IEEE Access
Multidisciplinary ⋮ Rapid Review ⋮ Open Access Journal

Received 4 February 2026, accepted 16 February 2026, date of publication 2 March 2026, date of current version 12 March 2026.

Digital Object Identifier 10.1109/ACCESS.2026.3669412

RESEARCH ARTICLE

Benchmarking Encoders and Self-Supervised Learning for Smartphone-Based Human Activity Recognition

GUSTAVO P. C. P. DA LUZ, DARLINNE H. P. SOTO, OTÁVIO O. NAPOLI, ANDERSON ROCHA, (Fellow, IEEE), LEVY BOCCATO, AND EDSON BORIN, (Member, IEEE)
Hub for Artificial Intelligence and Cognitive Architectures (H.IAAC), University of Campinas, Campinas 13083-852, Brazil

Corresponding author: Edson Borin (borin@unicamp.br)

This work was supported in part by the Ministry of Science, Technology, and Innovation of Brazil (MCTI), with resources granted by Federal Law 8.248 of October 23, 1991, through the Priority National Innovation Program administered by Softex (PPI-Softex) under Grant 01245.003479/2024-10; in part by the Coordination for the Improvement of Higher Education Personnel-Brazil (CAPES) under Grant 88887.999360/2024-00; in part by Brazilian National Research Council (CNPq) under Grant 315399/2023-6 and Grant 302458/2022-0; and in part by the São Paulo Research Foundation (FAPESP) under Grant 2013/08293-7 and Grant 2023/12865-8.

ABSTRACT Smartphone-based Human Activity Recognition (HAR) typically relies on deep learning models. However, performance varies with encoder architecture and the availability of labeled data. To address label scarcity, Self-Supervised Learning (SSL) exploits unlabeled data. However, existing benchmarks evaluate narrow combinations of encoders, SSL techniques, and refinement strategies, leaving the joint effects of encoder, SSL paradigm, and data availability underexplored under standardized conditions. We address this gap with a benchmark on the curated DAGHAR dataset, enabling reproducible evaluation across supervised and SSL settings from few-shot to full-data regimes. Across 11,232 trained models combining six encoders and four SSL techniques, we find that full fine-tuning consistently outperforms freezing, and that encoder architecture and SSL technique have comparable influence on accuracy, highlighting the importance of selecting effective combinations. While CNN-PFF with Time-Frequency Consistency (TF-C) achieves peak performance in most configurations, ResNet-SE-5 is the most robust overall encoder, delivering consistent results across datasets, SSL techniques, and data regimes, particularly in few-shot scenarios. Among the evaluated SSL techniques, TF-C demonstrates substantial data efficiency, with models reaching over 95% of peak accuracy using only 25-50 labeled samples per class, a performance level unattainable by supervised models. At full-data regimes, the best encoder varies by dataset, alternating between CNN-PFF and TS2Vec, while SSL, typically using TF-C or Learning from Randomness (LFR), can still outperform supervised learning. This standardized large-scale benchmark showed that SSL can outperform conventional supervised approaches with fewer labeled training data. In the future, it can be extended with new encoders, SSL techniques, and datasets.

INDEX TERMS Deep learning, human activity recognition (HAR), representation learning, self-supervised learning (SSL).

I. INTRODUCTION

Reliable and well-labeled data is not always available in human activity recognition (HAR) tasks, especially when data is collected from smartphones in unconstrained, real world environments. Since deep learning performance depends on both the model architecture and the amount of labeled data, this scarcity of labels remains a bottleneck for achieving robust HAR classifiers.

To address this limitation, recent studies have explored self-supervised learning (SSL) as an alternative to purely supervised training, as a means to mitigate annotation costs

The associate editor coordinating the review of this manuscript and approving it for publication was Md Kafiul Islam.

while enabling models to maintain high performance even with limited labeled data. SSL takes advantage of the abundance of unlabeled sensor data by introducing pretext tasks that allow pretraining without labels, later refined for downstream activity classification. However, choosing an appropriate combination of encoder architecture (also called backbone), SSL technique and refinement (fine-tuning) strategy remains challenging, as SSL lacks standardization and its benefits vary across datasets and encoder architectures [1], [2].

Previous work have partially examined these aspects. Supervised-oriented benchmarks [3], [4], [5] have compared different encoder architectures across datasets, providing valuable baselines, but ignored SSL and low-data regimes. On the other hand, SSL-oriented approaches [6], [7], [8], [9], [10], [11] have focused on individual techniques or specific encoder families, often restricted to one dataset and limited refinement strategies. As a result, the joint effects of encoder design, SSL technique, and refinement strategy under standardized evaluation remain largely unexplored.

The large-scale works most closely related to ours [2], [7], [12], [13] provide comparisons between SSL and supervised methods. However, they typically use a narrow set of encoders and SSL paradigms, leaving important combinations unexplored, particularly those widely used in smartphone-based HAR. Moreover, prior benchmarks often evaluate each SSL technique with its default encoder or restrict their analysis to a limited refinement strategy, which further constrains the understanding of how encoder architecture and SSL paradigms interact. The use of distinct datasets across studies also limits the comparability of their findings.

To fill this gap, this work presents a unified and large-scale benchmark that systematically evaluates multiple SSL and supervised methods across a curated set of encoder architectures and datasets. Building upon the DAGHAR benchmark dataset [5], we integrate it with SSL pretraining, few-shot learning, and a standardized experimental protocol. Using this protocol is crucial for a comparable evaluation of SSL in HAR, as stated by Logacjov [1].

We structured our analysis around the following research questions (RQs):

- **RQ1:** What is the best-performing encoder overall in supervised and SSL settings for HAR tasks?
- **RQ2:** How is the performance of encoders affected by the freeze refinement strategy?
- **RQ3:** What has a greater impact on final performance: changing the pre-training technique or the encoder architecture?
- **RQ4:** How does encoder performance vary across different target datasets?
- **RQ5:** What is the impact of the labeled data fraction on the performance of different encoders?
- **RQ6:** What is the minimum amount of labeled data required to achieve near-maximum performance, and what is the gain provided by using SSL in scarse data scenarios?

Beyond addressing six research questions, this benchmark provides a unified and reproducible setup for HAR evaluation, enabling direct comparison between supervised and SSL paradigms. It provides practical insights into how encoder choice, data availability, and pretraining strategies interact, guiding the design of efficient HAR models. The main contributions of this study are summarized as follows:

1) We identify the most effective encoders for both supervised and SSL settings: ResNet-SE-5 is the strongest overall encoder, while the CNN-PFF + Time-Frequency Consistency (TF-C) combination achieves the highest performance.
2) We show that full fine-tuning consistently outperforms frozen refinement, maximizing SSL gains across datasets, encoders, and data regimes.
3) We demonstrate that encoder architecture and SSL technique influence performance in a comparable manner, and that their interaction is a key determinant of downstream accuracy.
4) We analyze the role of dataset characteristics and label availability, showing that ResNet-SE-5 dominates under scarce labels, whereas TS2Vec leads when more labeled data are available, although TF-C with CNN-PFF remains the strongest specific configuration.
5) We show that SSL greatly reduces labeled data requirements overall, with TF-C-based models achieving the strongest gains by reaching over 95% of most dataset's peak accuracy with as few as 25-50 labeled samples per class.
6) We establish rigorous benchmarking practices for SSL in HAR, enforcing consistent preprocessing, controlled encoder architectures, fixed sequence lengths, unified metrics, and statistical comparisons. All code and experimental pipelines are publicly available on GitHub[1] to ensure full reproducibility.

The remainder of this paper is organized as follows: Section II reviews related work and contextualizes our contributions. Sections III and IV describe the encoder architectures and the selected SSL techniques, respectively. Section V presents the methodology, followed by results and discussion in Section VI. Finally, Section VII concludes the paper and outlines future research directions. At the end, we also have multiple Appendix Sections with complementary analysis of the research questions.

II. RELATED WORK

Using deep learning approaches for HAR classification tasks based on smartphone sensors has been extensively explored in the literature. Prior works can be broadly divided into two groups: Supervised-oriented approaches, which benchmark multiple encoders under fully supervised training, and SSL-oriented approaches, which investigate the benefits

[1] https://github.com/H-IAAC/benchmarking-encoders-ssl-har

of a pretraining without labels followed by a downstream classification task, reducing the amount of labels necessary to complete this task.

A. SUPERVISED-ORIENTED APPROACHES

Early approaches concentrated on assessing which encoder architectures perform best in HAR tasks when trained in a fully supervised manner. For example, Ronao and Cho [3] used a ConvNet for smartphone-based HAR using the University of California Irvine (UCI) HAR dataset at time and frequency domains and compared it to the performance of handcrafted features.

Subsequent studies such as Garcia-Gonzalez et al. [4] applied a similar strategy by testing combinations of Convolutional Neural Network (CNN), Long Short-Term Memory (LSTM), and hybrid architectures across the Real-life HAR dataset.

More recently, Napoli et al. [5] proposed the DAGHAR benchmark, a curated evaluation of multiple CNNs and Transformer-based encoders across six HAR datasets, establishing a supervised baseline.

In addition to these, numerous other studies have approached HAR from a fully supervised perspective [14], [15], [16], [17], [18], further reinforcing the maturity of this research direction. However, while these works offer valuable insights into encoder design, they do not investigate SSL techniques or scenarios involving limited labeled data. Moreover, the evaluated encoders are drawn from supervised learning paradigms, which limits the coverage of encoders that have shown strong performance but are traditionally explored in SSL settings.

B. SSL-FOCUSED APPROACHES

Another line of research has investigated self-supervised learning as a way to reduce annotation dependency by pretraining the encoder using unlabeled data at a pretext task in order to enhance HAR classifiers. Within SSL-oriented research, it is also possible to distinguish between small-scale studies that are focused on single datasets or limited encoder and SSL techniques sets, and large-scale benchmarks that cover multiple datasets and SSL techniques, usually with a smaller set of encoders.

1) SMALL-SCALE SSL

Several recent works have explored SSL for HAR in controlled, small-scale scenarios. Xu et al. [10] introduced Retrieval-Based Reconstruction (REBAR), comparing it against multiple techniques such as Temporal Neighborhood Coding (TNC), Contrastive Predictive Coding (CPC), Simple framework for Contrastive Learning of visual Representations (SimCLR), and Sliding-MSE using the TS2Vec encoder under freezing refinement. Sui et al. [11] proposed Learning from Randomness (LFR) and benchmarked it with several SSL methods with the standardized Time-Series representation learning framework via Temporal and Contextual Contrasting (TS-TCC) encoder under both fine-tuning and freezing refinements. Our previous work [6] compared TNC using two encoders, RNN (Recurrent Neural Network) and TS2Vec encoder, on the UCI HAR dataset. Hecker et al. [9] evaluated the TF-C technique with an TS-TCC encoder using full fine-tuning at the same dataset. Finally, da Silva et al. [8] investigated the CPC technique on multiple datasets (UCI HAR, MotionSense, RealWorld, and KuHAR) with a CNN encoder under freezing refinement. Together, these studies highlight the growing interest in SSL for HAR and reveal that despite their methodological diversity, most evaluations remain limited to a small number of datasets, encoder architectures, refinement strategy, and SSL techniques, leaving broader generalization and scalability questions open.

2) LARGE-SCALE SSL

Moving to large-scale works, benchmarks have aimed to compare multiple SSL paradigms across larger collections of datasets, SSL techniques and encoders. Haresamudram et al. [2] presented a benchmark covering multiple datasets and comparing various SSL paradigms under the freezing refinement strategy. Each classic SSL method adopted its default encoder, while supervised baselines included DeepConvLSTM, LSTM, Gated Recurrent Unit (GRU), CNN, and Multi-Layer Perceptron (MLP) models. The authors concluded that self-supervised techniques are data-efficient and robust to dataset imbalance, achieving comparable performance with fewer labeled samples, but generally do not surpass supervised models. Despite its breadth, the study focused mainly on earlier SSL paradigms and default encoder choices, omitting recent and specialized methods such as TNC, TF-C, Datum IndEx as Target (DIET), and LFR and the full fine-tuning refinement strategy. Moreover, it did not analyze the interaction between SSL techniques and encoder architectures, an aspect shown to be crucial for HAR performance, leaving open the systematic evaluation of modern SSL techniques and state-of-the-art encoders under standardized conditions, which is the focus of the present work.

The study of Qian et al. [12] provides a focused analysis of contrastive SSL methods for wearable based HAR. It compares several techniques, including Bootstrap Your Own Latent (BYOL), Simple Siamese Network (SimSiam), SimCLR, Nearest-Neighbour Contrastive Learning of visual Representations (NNCLR), and TS-TCC, across the UCI HAR, SHAR, and Heterogeneity Dataset for Human Activity Recognition (HHAR) datasets. Multiple encoder architectures were evaluated, such as DeepConvLSTM, LSTM, CNN, Autoencoder (AE), Convolutional Autoencoder (CAE), and Transformer, leading to the observation that CNN-based encoders generally outperform LSTMs and Transformers. The authors also remark that Residual Network (ResNet) architectures are commonly used as default encoders for contrastive methods. Although the work presents an extensive comparison of contrastive SSL models, it is limited to

full fine-tuning scenarios and does not include supervised learning.

Liu et al. [7] compared several SSL techniques, including FOCAL, TNC, MoCo, and SimCLR, on the RealWorld and PAMAP2 datasets under the freezing refinement strategy. Their experiments employed DeepSense and Swin Transformer as encoder architectures, allowing an analysis of both convolutional and transformer-based encoders. The results showed that SSL methods can provide competitive performance compared to fully supervised baselines, especially with the convolutional backbone under few-shot learning, but the study was limited to a small set of encoders not focused only at HAR and did not explore other SSL techniques widely used for HAR. Consequently, the influence of encoder design and refinement strategy on SSL performance in HAR remains only partially addressed.

Ek et al. [13] conducted a comparative study focused on adapting SSL methods from computer vision, SimCLR, Masked Autoencoder (MAE), and Data2Vec, to the wearable HAR domain. They employed both freezing and full fine-tuning refinements to assess the effect of downstream adaptation, and used two encoder architectures representing transformer and convolutional designs: HART and iSPLInception. Their results highlight that SSL can achieve competitive performance with reduced labeled data, but the relative effectiveness strongly depends on the encoder choice and refinement strategy. However, by concentrating on SSL techniques originally designed for images and evaluating them with only two encoder, the study leaves open questions about how SSL techniques, particularly those developed for time-series data, interact with a broader spectrum of encoders under a unified evaluation. This gap is addressed in the present work by benchmarking diverse encoder architectures and multiple modern SSL techniques on a standardized dataset.

Table 1 provides an overview of the most relevant related works, indicating for each study the HAR datasets used, whether SSL techniques and supervised learning was evaluated, the encoders used, the refinement strategy adopted, and whether low-data regimes were considered.

C. GAP IN THE LITERATURE

As summarized in Table 1, supervised-oriented approaches evaluate different encoders, but they require large amounts of labeled data and do not employ SSL techniques. In contrast, SSL-focused approaches explore pretraining strategies but usually with a smaller set of encoders and datasets. Few works analyze how SSL paradigms interact with encoder architectures and refinement strategies or evaluate their robustness under low-data regimes. Compared to the two most influential existing benchmarks these limitations become clearer. Haresamudram et al. [2] offer extensive coverage of HAR datasets but evaluate each classic SSL method only with its default encoder and restrict their analysis to freezing refinement. Ek et al. [13] examine both refinements strategies, yet their study is limited to two encoders and three SSL techniques originally designed for images, leaving unexplored the interaction between a broader set of encoders and modern SSL for time-series.

This work bridges these gaps by offering a unified and standardized benchmark that systematically compares state-of-the-art supervised and SSL techniques across six smartphone-based HAR datasets. It goes beyond evaluating a larger number of combinations by selecting complementary SSL techniques and encoder architectures originating from both supervised learning and SSL paradigms, all evaluated under a consistent experimental protocol with full fine-tuning and freezing refinements. Moreover, by extending the DAGHAR benchmark to include SSL training and few-shot scenarios, this study provides a more complete understanding of how data scale, encoder design, and pretraining strategy jointly affect HAR performance.

III. ENCODERS

Finding effective encoder architectures is important in HAR because the quality of learned representations is directly impacted by the choice of encoder. Different encoders capture distinct patterns in the sensor signals, which can influence the performance of both supervised and self-supervised models. While some encoders achieve strong performance in fully supervised settings, others are particularly suitable for SSL pretraining, providing representations that can later be refined for classification with limited labeled data. Evaluating a diverse set of encoders enables a systematic understanding of their contribution to model accuracy, robustness, and label efficiency.

We selected six encoder architectures for this study, combining state-of-the-art encoders commonly used for supervised HAR [5] (ResNetSE-5, CNN-PFF, IMU Transformer) with encoders extracted from SSL techniques (TS2Vec Encoder, TS-TCC Encoder, RNN), allowing us to compare strong and traditionally used models across multiple deep learning families. These encoders span various paradigms, including base convolutional networks, residual networks, dilated convolutions, recurrent networks, and a Transformer-based architecture. A particular focus was placed on convolutional encoders, as they have demonstrated superior performance in SSL for HAR tasks [7], [12]. Fig. 1 provides a visual overview of the encoder architectures employed, highlighting the diversity of representation learning strategies evaluated in this work.

A. ResNet-SE-5 ENCODER

ResNet is a widely used CNN originally proposed for the image domain [19], where the key idea was to learn residual functions with reference to the layer inputs, enabling the training of substantially deeper models without degradation of accuracy.

For sequential signals, authors have proposed adaptations of ResNet to 1D time-series data [20], sometimes enhanced with Squeeze-and-Excitation (SE) mechanisms. As this

TABLE 1. Related work on smartphone sensor-based HAR, highlighting evaluated datasets, SSL usage, encoders, supervised usage, fine-tuning strategies, and handling of low-data regimes. The top half emphasizes Supervised-oriented approaches, while the lower half highlights SSL-oriented approaches with small and large-scale experimentation. Most prior SSL studies are limited to a small number of datasets, encoders, and different evaluation protocols. Our work extends this by systematically benchmarking modern SSL techniques and encoders for HAR under standardized conditions and low-data regimes.

Work	Evaluated HAR Datasets	SSL Techniques	Encoders	Supervised	Refinement Strategy	Low-Data
Supervised-Oriented Approaches						
Ronao and Cho [3] (2016)	UCI HAR	✗	ConvNet	✓	Full Fine-tuning	✗
Garcia-Gonzalez *et al.* [4] (2023)	Real-life HAR	✗	DS-CNN, LSTM, Bi-LSTM, (DS-CNN)-LSTM, (DS-CNN)-(Bi-LSTM)	✓	Full Fine-tuning	✗
Napoli *et al.* [5] (2024)	DAGHAR (UCI HAR, RealWorld Waist, RealWorld Thigh, MotionSense, KuHar, WISDM)	✗	CNN-PFF, CNN-PF, CNN 1D, CNN 2D, ConvNet, IMU CNN, MLP, ResNet, ResNetSE, ResNetSE-5, IMU Transformer	✓	Full Fine-tuning	✗
SSL-Oriented Approaches						
Small-Scale						
Xu *et al.* [10] (2023)	UCI HAR	REBAR, TNC, TS2Vec, CPC, SimCLR, Sliding-MSE	TS2Vec Encoder	✓	Freeze	✓
Sui *et al.* [11] (2024)	UCI HAR	LFR, DIET, TS-TCC, SimCLR, SimSiam, Autoencoder, DACL, STab, SCARF, Random Init	TS-TCC Encoder	✓	Full Fine-tuning and Freeze	✓
da Luz *et al.* [6] (2024)	UCI HAR	TNC	RNN, TS2Vec Encoder	✗	Freeze	✗
Hecker *et al.* [9] (2024)	UCI HAR	TF-C	TS-TCC Encoder	✓	Full Fine-tuning	✓
da Silva *et al.* [8] (2024)	UCI HAR, MotionSense, RealWorld, KuHar	CPC	CNN	✓	Freeze	✓
Large-Scale						
Haresamudram *et al.* [2] (2022)	Capture-24, HHAR, MyoGym, Wetlab, MobiAct, MotionSense, USC-HAD, Daphnet FoG, MHEALTH, PAMAP2	Multi-task Self-supervision, Masked Reconstruction, CPC, Autoencoder, SimCLR, SimSiam, BYOL	Default per SSL technique + supervised baselines with DeepConvLSTM, LSTM, GRU, CNN, MLP	✓	Freeze	✓
Qian *et al.* [12] (2022)	UCI HAR, SHAR, HHAR	BYOL, SimSiam, SimCLR, NNCLR, TS-TCC	DeepConvLSTM, LSTM, CNN, AE, CAE, Transformer	✗	Full Fine-tuning	✓
Liu *et al.* [7] (2023)	RealWorld, PAMAP2	FOCAL, TNC, SimCLR, MoCo, CMC, MAE, Cosmo, Cocoa, MTSS, TS2Vec, GMC, TS-TCC	DeepSense, SW-T	✓	Freeze	✓
Ek *et al.* [13] (2025)	UCI HAR, MotionSense, RealWorld, HHAR, MobiAct, PAMAP2	SimCLR, MAE, Data2vec	HART, iSPLInception	✓	Full Fine-tuning and Freeze	✓
This Work	DAGHAR (UCI HAR, RealWorld Waist, RealWorld Thigh, MotionSense, KuHar, WISDM)	TNC, TF-C, DIET, LFR	ResNetSE-5, CNN-PFF, IMU Transformer, TS2Vec Encoder, TS-TCC Encoder, RNN	✓	Full Fine-tuning and Freeze	✓

encoder obtained one of the highest performances at the DAGHAR paper [5], we chose the ResNet-SE-5 Encoder.

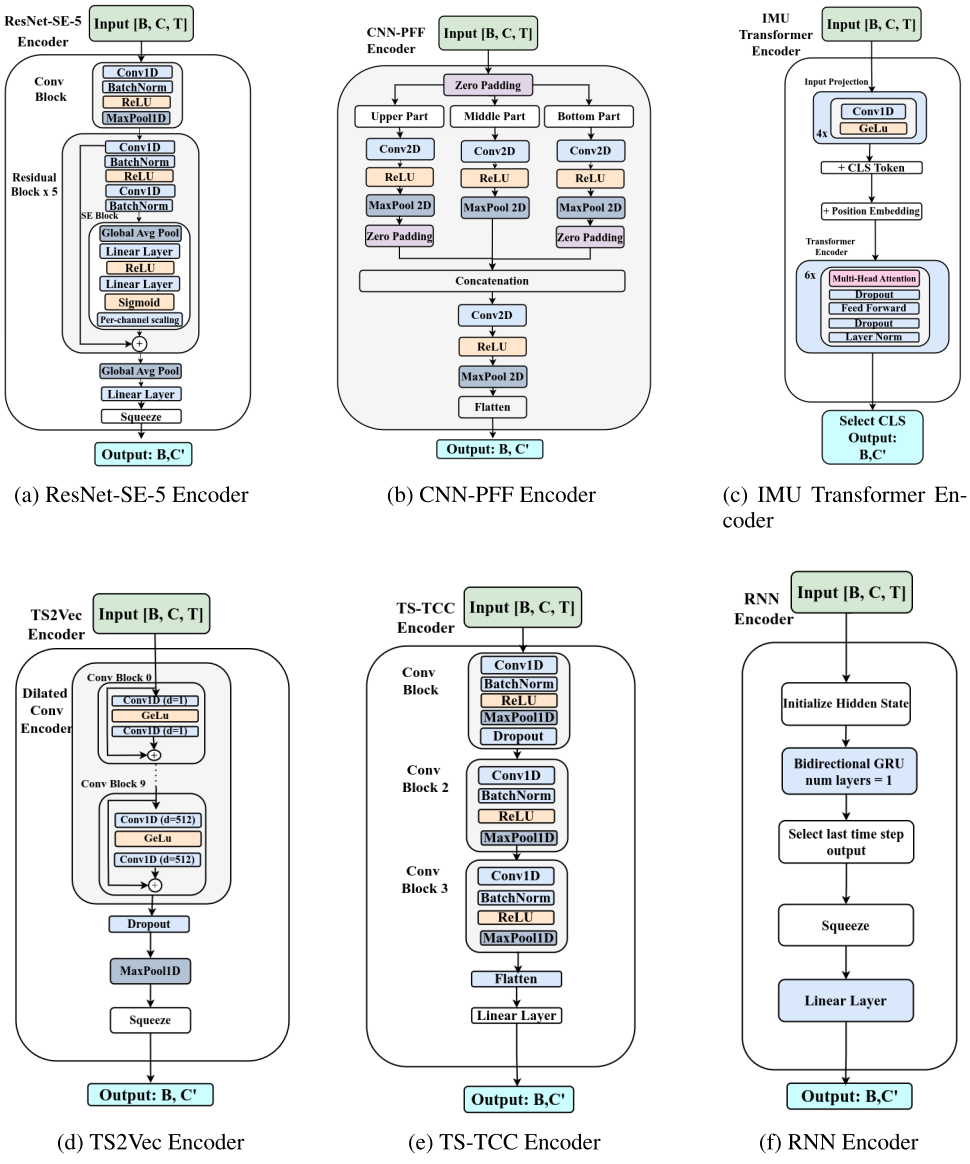
The ResNet-SE-5 architecture proposed by Mekruksavanich et al. [20] is illustrated in Fig. 1a, composed of an initial convolutional block followed by a sequence of five residual blocks.

Each residual block consists of two convolutional layers with batch normalization and a Rectified Linear Unit (ReLU) activation, combined with a SE block and a residual connection. The SE block consists in a global average pooling layer followed by two linear layers with a ReLU activation, followed by a sigmoid and scaling per channel, where each channel in the original feature map is multiplied by its learned weight.

The residual connection adds the input tensor to the output of the block, allowing the block to refine the input rather than relearn it entirely. The output of the stacked blocks is aggregated through a global average pooling layer, linear layer and squeezing, which reduces the temporal dimension into a compact feature vector of size 64, considering the input of 6 channels and 60 time steps.

B. CNN-PFF ENCODER

The CNN-PFF encoder was originally introduced by Ha and Choi [21] for time-series feature extraction. Unlike traditional CNNs, this architecture employs partial weight sharing, where different kernels are applied to distinct input channel groups. The authors incorporate zero padding to



(a) ResNet-SE-5 Encoder (b) CNN-PFF Encoder (c) IMU Transformer Encoder

(d) TS2Vec Encoder (e) TS-TCC Encoder (f) RNN Encoder

FIGURE 1. Encoder architectures evaluated in this study. The selected models include state-of-the-art encoders commonly used for supervised HAR (ResNet-SE-5, CNN-PFF, IMU Transformer) and encoders derived from SSL techniques (TS2Vec, TS-TCC, RNN), providing a diverse set of deep learning paradigms. All encoders receive input tensors in the format Batch × Channels × Timesteps (B, C, T) and produce a feature representation of shape Batch × Features (B, C').

avoid interference between signals from different sensors during convolution.

In this work, we adopted the full CNN-PFF configuration with three processing parts, as illustrated in Fig. 1b: upper, middle, and lower parts. Each part applies convolution, ReLU activation, and max pooling, where the upper and lower part applies zero padding again. The outputs of these parallel parts are then concatenated and passed through a shared convolutional block, which integrates information across the groups. This convolutional block is followed by ReLU activation, max pooling, and flattening to obtain the final feature representation of dimension $64 \times 12 = 768$, considering the input of 6 channels and 60 time steps.

This architecture demonstrated high performance with a smaller number of parameters compared to other 1D-CNNs, while being more efficient in capturing inter-sensor relationships than 2D-CNNs [21]. Our choice to include CNN-PFF in our study was also motivated by its strong performance in the standardized view of the benchmark dataset DAGHAR [5].

C. IMU TRANSFORMER ENCODER

Beyond CNNs and RNNs, transformer-based architectures have recently gained popularity for modeling time series data, following their success in natural language processing

and computer vision. Specifically for inertial-based human activity recognition, Shavit and Klein [22] introduced a Transformer encoder designed to map sequences of Inertial Measurement Unit (IMU) data into a latent feature space.

As illustrated in Fig. 1c, the model begins with a series of four 1D convolutions, each followed by a Gaussian Error Linear Unit (GeLU) activation, which project the raw IMU signal into a higher dimensional latent representation. A learnable class token is then added to this sequence, and positional embeddings are added forming the input to the transformer encoder.

The Transformer encoder itself follows the standard design proposed by Vaswani et al. [23], stacking 6 encoder layers. Each layer consists of a Multi-Head self-Attention (MHA) mechanism with 8 heads, a feedforward block, dropout regularization, and Layer normalization.

The output corresponding to the class token position is used as the aggregated sequence representation. This vector can be further processed by a classifier head. In our implementation, we adopt the configuration of Shavit and Klein [22], setting a 128 feedforward dimension, resulting in a 64 latent dimension, 0.1 for dropout, and GeLU as the activation.

D. TS2Vec ENCODER

The TS2Vec encoder corresponds to the architecture originally introduced in TS2Vec [24], an SSL technique for time series representation learning. It is composed of a dilated convolution encoder, with a stack of dilated 1D convolutional blocks, where the dilation rate increases exponentially with network depth. This design enables the receptive field to expand fast, allowing the model to capture both short-term and long-term temporal dependencies without requiring recurrent or attention-based mechanisms.

As illustrated in Fig. 1d, the dilation rate doubles at each successive block, starting from 2^0 up to 2^n. In our setup, we employ the default proposed depth of 10 convolutional blocks with hidden dimension 64, resulting at a maximum receptive field of $2^9 = 512$ time steps. Each block applies a sequence of convolution, GeLU activation, and residual connection, followed by dropout regularization.

At the end of the dilated convolutional stack, the encoder applies dropout and a max-pooling operation across the temporal dimension, where the pooling kernel size k is set equal to the sequence length. This operation aggregates information over the full sequence, producing a compact feature representation of dimension 320.

This encoder has become a standard choice in recent works on time-series representation learning, being adopted not only in TS2Vec but also in subsequent methods such as REBAR [10] for multiple SSL techniques, which influenced in the choice of the dimensions used in this work.

E. TS-TCC ENCODER

The TS-TSS Encoder is a convolutional encoder originally introduced in 2021 by TS-TCC [25], a SSL technique for time series. Similarly to TS2Vec encoder, we extracted only their encoder for our benchmark. This architecture is based on the classical Fully Convolutional Network (FCN) for time series classification proposed by Wang et al. [26] in 2017.

Fig. 1e shows the architecture of the encoder, which is composed of three standard convolutional blocks. Each block consists of a 1D convolution, batch normalization, ReLU activation, and max pooling. A dropout layer is additionally applied after the first block to reduce overfitting. After the convolutional stack, the representation is flattened and passed through a linear layer to obtain the final feature representation with an output dimension of 2304.

This structure is also the default encoder for three SSL techniques explored in this study: TF-C, LFR, and DIET, which provided the basis for the parameter setup we adopted and resulted in a dimensionality of 2304.

F. RNN ENCODER

Recurrent Neural Networks (RNNs) are widely applied to time series sensor data due to their ability to maintain contextual information across sequential inputs [20]. Some approaches even combine RNNs and CNNs into hybrid classifiers to benefit from both temporal and spatial feature extraction. In our work, we adopt the RNN architecture proposed by Tonekaboni et al. [27], which serves as the default encoder for TNC.

This encoder is based on Gated Recurrent Units (GRUs), which are designed to capture long range temporal dependencies through gated sequential processing. As illustrated in Fig. 1f, the model initializes a hidden state and processes the sequence with the bidirectional GRU. From the resulting sequence of hidden states, the output corresponding to the last time step is selected, squeezed, and passed through a linear projection layer to produce the final feature representation.

We adopt the same configurations from Tonekaboni et al. [27], with one recurrent layer of hidden size 100, applied to input sequences of 6 channels. Because the GRU is bidirectional, the hidden states are concatenated, yielding $2 \times 100 = 200$ features per timestep. A final linear projection expands this to an encoding size of 320, which was based at the encoding size of previous works [6], [10] with TNC at raw time series data, differently than the original one, which was analyzed for HAR but with handcrafted features.

IV. SELF-SUPERVISED LEARNING TECHNIQUES

HAR datasets collected from smartphones and wearable sensors are often limited in labeled samples, while large amounts of raw sensor data are readily available. SSL takes advantage of this scenario and aims to learn meaningful representations from unlabeled data, reducing the dependence on manual annotation. SSL pretext tasks allow models to capture temporal and frequency patterns inherent in motion signals, improving downstream classification performance, particularly in low-data or few-shot scenarios. However, SSL performance strongly depends on the choice of pretext task,

encoder architecture, and refinement strategy, and not all SSL techniques achieve good results in every setting.

We investigated four SSL techniques for pretraining our HAR models: Temporal Neighborhood Coding (TNC) [27], Time-Frequency Consistency (TF-C) [28], Learning From Randomness (LFR) [11], and Datum IndEx as Target (DIET) [29]. These four techniques were chosen to reflect complementary representation learning strategies, including classical contrastive and modern assumption-based approaches that have demonstrated effectiveness in HAR.

First, contrastive SSL methods, known to achieve strong HAR performance [12], include TNC, which uses sampling contrast and has shown good results in HAR [6], [7], [10], and TF-C, which relies on augmentation contrast to capture both temporal and frequency information, achieving a high performance on UCI HAR [9]. Second, simpler modern assumption-based techniques, LFR [11] and DIET [29], provide promising performance while requiring minimal pretext design. Together, these four techniques allow a systematic evaluation of how pretext task design interacts with encoder choice and refinement strategy across datasets and data regimes.

A. TNC

Temporal Neighborhood Coding (TNC) was introduced by Tonekaboni et al. [27] in 2021 as contrastive learning SSL technique. The core idea is that temporally neighboring windows of a sequence should share similar latent representations, while sufficiently distant windows should differ. The technique was proposed for the medical domain with Electrocardiogram (ECG) signals, but was also evaluated for HAR at the original paper and on further works [6], [10], both of them at the UCI dataset.

To define positive pairs, the original work employed the Augmented Dickey-Fuller (ADF) statistical test, although later variants also adopt simpler similarity metrics such as cosine similarity [30]. Negative pairs are sampled from windows that are randomly chosen while ensuring a sufficient temporal distance. We adopted the cosine similarity in this work, as it provides a lower training time with no significant impact in performance, as verified in our previous work [6].

The original encoder of TNC is the RNN described in the last section. Latest works [6], [10] discovered that replacing the RNN with the TS2Vec encoder results in higher performance. As projection head, we adopted the standard proposed MLP architecture of a Fully Connected (FC) layer followed by a ReLU activation, a dropout layer, and another FC layer. This projection head acts as a binary classifier that predicts if each selected window is close or distant.

The training objective employs a binary cross-entropy loss combined with a sigmoid layer, where query windows are compared against both close positive and distant negative samples. For negative pairs, a weighting factor accounts for the probability that a randomly chosen distant sample might still belong to the same semantic class. While prior works experimented with values of 0.05 and 0.2 for this parameter, we fix it to 0.2 as this setting has shown the most consistent improvements [6], [10].

B. TF-C

Time-Frequency Consistency (TF-C) was proposed by Zhang et al. [28] in 2022 as a contrastive SSL technique that explores representation of the time and frequency domain. The original proposal evaluated TF-C for different time-series tasks, such as epilepsy, sleeping and hand gesture. Subsequent works evaluated TF-C for HAR at the UCI dataset [9], [31].

Essentially, the pretext task corresponds to distinguish positive pairs from negative pairs in the joint time-frequency-consistency space. The architecture of TF-C consists of three pipelines: time, frequency, and consistency. The time and frequency pipelines each employ the three-block TS-TCC convolutional encoder illustrated in Fig. 1e. While Guo et al. [31] experimented with a 1D-ResNet encoder, their results showed that the TS-TCC encoder achieved superior performance, in line with Hecker et al. [9].

In TF-C, each encoder is followed by a projector at the consistency pipeline, implemented as a linear layer, batch normalization, ReLU activation, and a final linear layer. Unlike other SSL methods that have a projection head as separated components, these projectors are inherent components to the TF-C architecture rather than auxiliary modules, since they map encoder outputs into a shared feature space. Each projector produces a 128 dimensional representation, and the outputs of the time and frequency branches are concatenated, forming a 256 dimensional feature vector that serves as the basis for downstream classification. Training is driven by the Normalized Temperature-Scaled Cross Entropy (NT-Xent) contrastive loss, applied across augmented views of the input. The loss is decomposed into three complementary components corresponding to the time, frequency, and consistency objectives.

Unlike approaches that operate on a standalone encoder with an auxiliary projection head, TF-C imposes a rigid architectural structure. As discussed by Hecker et al. [9], applying TF-C means training a composite backbone comprising two copies of the encoder network named as G_T and G_F (processing the time and frequency domains), two projectors named as R_T and R_F, and FFT operations for frequency analysis. As a result, downstream evaluation necessarily relies on this integrated architecture rather than on an isolated encoder. This increased architectural complexity, while often effective, introduces additional components and parameters that are not present in other SSL techniques designed for single-encoder settings. Consequently, performance differences observed under TF-C should be interpreted in light of this composite backbone configuration.

C. LFR

Learning From Randomness (LFR) is an SSL technique proposed by Sui et al. [11] in 2023 and states that good data

representation can be learned by training a model to predict the outputs of multiple random data projections. This allows an approach that does not rely on specific augmentations or masking, being suitable for different encoders.

The technique relies on three components: an encoder, K random projectors, and K predictors. Each projector produces a pseudo-target, which the corresponding predictor must approximate from the encoder representation. The random projectors are frozen, forcing the encoder and the predictors to be refined. The authors proposes selecting a subset of projectors that maximizes variability between them.

The neural network architecture of the original paper depends on the data domain, thus we will focus on the architecture that was evaluated on the UCI dataset for HAR and for other time-series classification problems. The TS-TCC encoder was used as the representation model. For the predictors a single linear layer was used, and for the projectors the authors proposed using two convolutional blocks followed by linear layers. The adopted loss is the custom BatchWise Barlow Twins loss specifically designed to measure differences between the projector and the predictor.

D. DIET

Proposed by Balestriero [29] in 2023, the Datum IndEx as Target (DIET) technique treats each sample as its own class, determined by its Datum Index (the position of the sample within the dataset). DIET was originally proposed as a SSL technique for image classification achieving the best performance when paired with ResNet variants. The technique was first used for HAR and time-series at the work of LFR [11], which evaluated it for HAR at the UCI dataset using the TS-TCC encoder and also to different data domains, including images, time-series, and tabular data. We follow the original work and use cross-entropy loss with label smoothing of 0.8.

V. MATERIALS AND METHODS

In this section, we describe the datasets employed in the work and the methodology designed to address the research questions following a standardized experimental protocol.

A. DATASETS DESCRIPTION

To choose the datasets used in this work, diverse options could be considered. We selected the standardized view of the curated benchmark dataset for HAR tasks DAGHAR [5], which contains accelerometer and gyroscopes data from smartphones sensors placed among different locations of the body. Some advantages of using this dataset are that it is curated from over 40 HAR datasets options, based on availability of raw data, data integrity, and support of a broad range of regular human activities, being balanced in terms of classes per dataset.

Another advantage of using DAGHAR is that the data is standardized in terms of activities, containing up to six activities of sit, stand, walk, walk upstairs, walk downstairs, and run. We used all datasets available at DAGHAR: KuHar (KH), MotionSense (MS), RealWorld Waist (RW-Waist), RealWorld Thigh (RW-Thigh), UCI HAR, and WISDM. The data is also standardized as follows: The sampling rate is 20 Hz and the gravity component was removed. The time-series of each sensor channels were sliced into non-overlapping 3 seconds windows, and partitioned into train, validation and test using the ratio 70/20/10, ensuring that different users fall into each subset. This was done in order to report results that reflect cross-user generalization and prevent the memorization of specific patterns of individual subjects.

B. OVERVIEW OF THE EXPERIMENTAL PIPELINE

Fig. 2 summarizes our experimental design, which comprises a supervised baseline and a SSL pipeline, both converging on a common refinement and evaluation procedure.

The supervised baseline trains models directly on labeled train/val partitions data, whereas the SSL pipeline first performs pre-training on unlabeled train/val partitions of the target dataset using four different SSL techniques, followed by a refinement step with labeled fractions of the same dataset partitions for few-shot evaluation.

Both paradigms use the same set of encoders and the same classification prediction heads during fine-tuning and evaluation, ensuring comparability; the input dimensions are slightly adjusted according to the encoder architecture. The experimental workflow is organized into three evaluation stages. The first one is the set of models trained with a baseline supervised training. The second one is the set of models trained with SSL with a downstream refinement stage. The third stage is to compare all of the performances across different configurations to answer the research questions.

C. MODEL COMPONENTS: ENCODERS, PROJECTION HEADS, AND PREDICTION HEADS

As remarked in Section III, we evaluated six encoder architectures: ResNet-SE-5, CNN-PFF, IMU Transformer, TS2Vec Encoder, TS-TCC Encoder, and RNN. In the SSL setting, encoder features are passed to a projection head, which implements the specific transformation space required by each SSL technique and follows the configuration proposed by the original authors.

In both SSL and supervised training, the final classification is carried out by a prediction head, which consists of a MLP mapping the encoder output to the label space, consistent with common practices in HAR model training. Haresamudram et al. [2] showed that the most frequently used non-linear head in existing HAR literature at more than 20 works is the MLP. To standardize the prediction head across all models, we used a fixed architecture consisting of an input layer that matches the encoder output dimensionality with a ReLU activation, a hidden layer of 128 units with a ReLU activation, and a fixed six-unit output layer corresponding to the maximum activity classes in DAGHAR datasets.

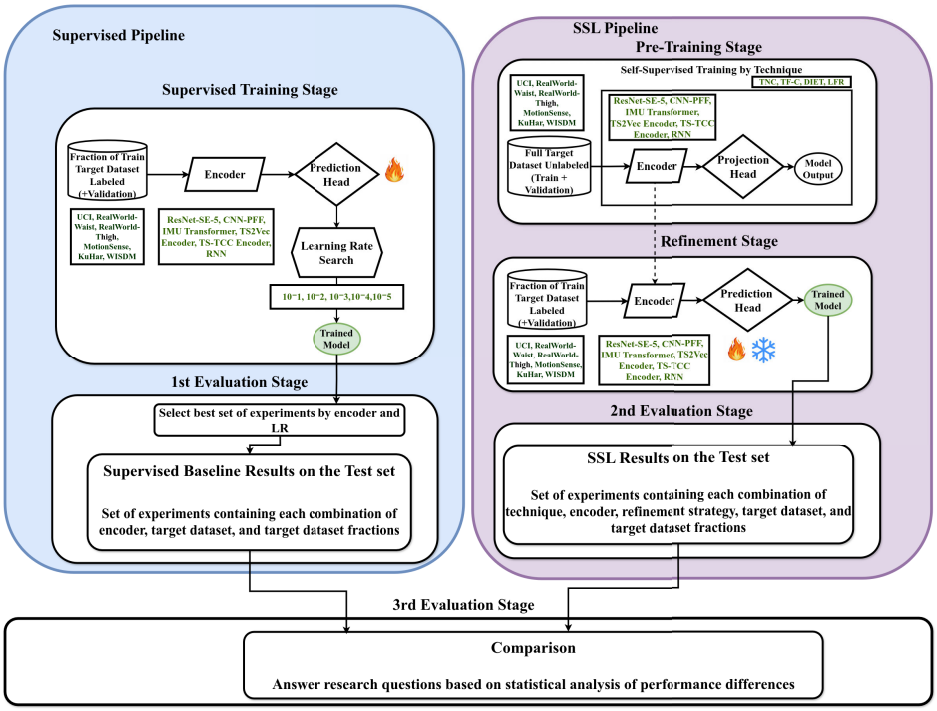


FIGURE 2. Overview of the experimental methodology combining Supervised and SSL pipelines. The Supervised Pipeline at the left trains each encoder directly on labeled fractions of the target datasets, using a learning rate search to establish the baseline for each encoder. The SSL Pipeline at the right first performs pre-training on the unlabeled target dataset using one of four SSL techniques (TNC, TF-C, DIET, LFR), followed by refinement on labeled fractions through either frozen or fully fine-tuned encoders. Both pipelines share the same encoders (ResNetSE-5, CNN-PFF, IMU Transformer, TS2Vec Encoder, TS-TCC Encoder, RNN) and prediction heads to ensure fair comparison. The workflow comprises three evaluation stages: supervised baseline, SSL results, and statistical comparison, covering six datasets.

TABLE 2. Encoders, projectors, and prediction heads configurations sorted by computational complexity. MACs and number of parameters are reported in millions (M). *The backbone of TF-C comprises two copies of the encoder, two projectors, and a FFT operation. For all the other training strategies, the backbone is comprised by a single encoder.

Encoder	Projector (MLP)	Prediction Head (MLP)	Number of Parameters (M)			MACs (M)
			Backbone*	Pred. Head	Total Params	
Supervised, DIET, TNC, LFR						
CNN-PFF	–	$(768 \to 128 \to 6)$	0.05	0.10	0.15	1.67
ResNet-SE-5	–	$(64 \to 128 \to 6)$	0.13	0.01	0.14	3.07
RNN	–	$(320 \to 128 \to 6)$	0.13	0.04	0.17	4.08
TS-TCC Encoder	–	$(2304 \to 128 \to 6)$	3.04	0.30	3.33	5.07
IMU Transformer	–	$(64 \to 128 \to 6)$	0.22	0.01	0.23	6.97
TS2Vec Encoder	–	$(320 \to 128 \to 6)$	0.64	0.04	0.68	38.16
TF-C						
FFT + 2× CNN-PFF	$2 \times (768 \to 128)$	$(256 \to 128 \to 6)$	0.55	0.03	0.59	3.65
FFT + 2× ResNet-SE-5	$2 \times (64 \to 128)$	$(256 \to 128 \to 6)$	0.35	0.03	0.39	6.27
FFT + 2× RNN	$2 \times (320 \to 128)$	$(256 \to 128 \to 6)$	0.49	0.03	0.52	8.34
FFT + 2× TS-TCC Encoder	$2 \times (2304 \to 128)$	$(256 \to 128 \to 6)$	7.32	0.03	7.35	10.83
FFT + 2× IMU Transformer	$2 \times (64 \to 128)$	$(256 \to 128 \to 6)$	0.54	0.03	0.57	14.06
FFT + 2× TS2Vec Encoder	$2 \times (320 \to 128)$	$(256 \to 128 \to 6)$	1.51	0.03	1.54	76.50

Table 2 details the configurations of the models, including encoders, projectors, and prediction heads, the number of parameters, and the Multiply-Accumulate operations (MACs) per inference. It is possible to see that we are considering large and small encoders, from 46k to more than 3M parameters, which also impacts the number of parameters in the prediction head.

For SSL techniques such as TNC, DIET or LFR, the encoder features are passed through projection heads during pre-training, but these heads are discarded before fine-tuning, leaving only the encoder plus a prediction head. On the other hand, TF-C requires its projectors to remain attached to the encoders. To ensure fair comparison across encoders, we retained these projectors and standardized

TABLE 3. Few-shot sampling strategy overview. Each cell shows total samples with the corresponding dataset percentage in parenthesis. The max column displays full dataset sizes.

	Samples per Class							
Dataset (Classes)	1	5	10	25	50	100	200	max
KuHar (6)	6 (0.4%)	30 (2.2%)	60 (4.3%)	150 (10.8%)	300 (21.6%)	600 (43.1%)	1200 (86.2%)	1392 (100%)
MotionSense (6)	6 (0.2%)	30 (0.8%)	60 (1.7%)	150 (4.2%)	300 (8.4%)	600 (16.9%)	1200 (33.7%)	3558 (100%)
RealWorld-Thigh (6)	6 (0.1%)	30 (0.6%)	60 (1.2%)	150 (2.9%)	300 (5.8%)	600 (11.6%)	1200 (23.2%)	10338 (100%)
RealWorld-Waist (6)	6 (0.1%)	30 (0.6%)	60 (1.2%)	150 (2.9%)	300 (5.8%)	600 (11.6%)	1200 (23.2%)	10332 (100%)
UCI (5)	5 (0.2%)	25 (1.0%)	50 (2.1%)	125 (5.2%)	250 (10.3%)	500 (20.7%)	1000 (41.3%)	2420 (100%)
WISDM (4)	4 (0.05%)	20 (0.2%)	40 (0.5%)	100 (1.1%)	200 (2.3%)	400 (4.6%)	800 (9.1%)	8748 (100%)

their outputs as stated in TF-C: each projector maps its encoder features to 128 dimensions, which are concatenated to form a 256 dimensional vector. This representation is then passed to the same MLP prediction head used for all encoders (256→128→6). In this way, TF-C remains faithful to its original design while still aligning with our evaluation protocol with the proposed original dimensions of the encoders detailed in Section III. Appendix E, discusses the impact of each architecture on inference time.

D. SUPERVISED PIPELINE BASELINE

As a baseline, each encoder was trained from scratch (random initialization) and directly on labeled data. Since it would not make sense to freeze random weights, only the full fine-tuning strategy was applicable. Training was performed for up to 100 epochs with early stopping based on validation loss (patience of 50 epochs) and a batch size of 64.

To evaluate performance under varying label availability, we applied a stratified sampling approach that creates subsets with fixed numbers of samples per class. For each dataset, we generated training sets containing 1, 5, 10, 25, 50, 100, and 200 samples per class, plus the full dataset (maximum available samples). The mapping between samples per class and data percentage and the values for each dataset are shown at Table 3. It is possible to notice that datasets have different sizes, as 200 samples per class corresponds to 86.2% of KuHar, while it is only 9.1% of WISDM. This shows the importance of the stratification employed, ensuring that similar quantities of samples are being used during few-shot learning.

The sampling algorithm operates by first deterministically shuffling all dataset indices using a seed, then grouping indices by class label while maintaining the shuffled order. For each class, the first N samples are selected from this sequence, where N is the target samples per class. We repeated the experiments three times, using different seeds for each repetition (42, 43 and 44), which leads to distinct but reproducible training subsets.

This cumulative sampling approach ensures that when moving from 1 to 5 samples per class, the first sample remains, so that the 5-sample set includes that sample plus four additional ones. This was done to allow progressive evaluation where each larger subset naturally extends the smaller ones, simulating a realistic scenario where labeled data is incrementally collected over time. We also use the additional shuffle of the dataloader to compose training batches after the subset selection.

TABLE 4. Hyperparameters for SSL pre-training across different techniques.

SSL Tech.	Drop Last	Optimizer	Learning Rate	Weight Decay	Betas
TNC	False	AdamW	10^{-5}	✗	✗
TF-C	True	Adam	3×10^{-4}	3×10^{-4}	$(0.9, 0.99)$
DIET	False	Adam	3×10^{-4}	3×10^{-4}	$(0.9, 0.99)$
LFR	False	Adam	3×10^{-4}	3×10^{-4}	$(0.9, 0.99)$

We conducted a search over learning rates to select the optimal value for each encoder considering the supervised training among the considered data sampling. We found that the best learning rate for all encoders was 10^{-4} except the TS2Vec encoder, which achieved better performances with the learning rate of 10^{-3}. The complete details of this search with results is available at the Appendix A. For each encoder, the experiments using the learning rate that produced the best overall performance were selected to define the supervised baseline for SSL comparisons.

E. SSL PRE-TRAINING STAGE

The SSL pre-training stage used the unlabeled training and validation partitions of the target dataset and was performed separately for each encoder and dataset combination, and for three independent runs. Encoders were connected to the projection heads defined by each technique, which were trained without early stopping, keeping the last checkpoint for refinement. This choice allowed the use of more available unlabeled data, including validation splits, increasing sample diversity.

The data module used at pre-training employs sliding windows that preserves the original temporal structure, with exception to the TNC module, where training and validation sequences from all individuals are truncated to a uniform length of timesteps, as designed by Tonekaboni et al. [27].

The hyperparameters of pre-training are summarized at Table 4, following the proposal of each technique but setting a standardized number of 100 epochs and batch size of 64 for a fair evaluation.

F. SSL REFINEMENT STAGE

After pre-training, models were refined on labeled data using two strategies: (*i*) freeze, where the encoder remains unchanged and only the prediction head is adjusted, and

TABLE 5. Supervised learning ablation study (total = 4,320 models).

Field	Value Counts (n = 4,320)
Learning Rate	10^{-1} (864), 10^{-2} (864), 10^{-3} (864), 10^{-4} (864), 10^{-5} (864)
Encoder	CNN-PFF (720), ResNet-SE-5 (720), TS-TCC Encoder (720), IMU Transformer (720), RNN (720), TS2Vec Encoder (720)
Target Dataset	KH (720), RW-Waist (720), MS (720), UCI (720), WISDM (720), RW-Thigh (720)
Samples per Class	1 (540), 5 (540), 10 (540), 25 (540), 50 (540), 100 (540), 200 (540), max - 100% of the dataset (540)

TABLE 6. Main benchmark including SSL experiments and selected supervised runs (total = 7,776 models).

Field	Value Counts (n = 7776)
Training Method	TNC (1728), TF-C (1728), DIET (1728), LFR (1728), Supervised (864)
Encoder	CNN-PFF (1296), ResNet-SE-5 (1296), IMU Transformer (1296), RNN (1296), TS2Vec Encoder (1296), TS-TCC Encoder (1296)
Target Dataset	KH (1296), RW-Waist (1296), MS (1296), UCI (1296), WISDM (1296), RW-Thigh (1296)
Samples per Class	1 (972), 5 (972), 10 (972), 25 (972), 50 (972), 100 (972), 200 (972), max - 100% of the dataset (972)
Fine-tuning	Full Fine-tune (4320), Freeze (3456)

(*ii*) full fine-tuning, where the entire model is refined. At this stage training used a batch size of 64, up to 100 epochs, and early stopping with a 50 epoch patience on validation loss. Each experiment was repeated three times with different seeds. We chose to pre-train and refine the encoders based on the same target dataset.

To assess robustness on few-shot learning, datasets were fractioned by the same method employed at the supervised pipeline described in Section V-D. The learning rate chosen for refinement was also 10^{-4} for all encoders and 10^{-3} for the TS2Vec encoder, aligned with the method of supervised learning rate search.

G. EVALUATION

The experimental workflow was structured into three evaluation stages, where in total 11,232 models were trained across supervised and SSL pipelines. The first stage focused on the supervised baseline, including a learning rate ablation study. In total, 4,320 supervised models were trained for the ablation study outlined in Table 5 with results exhibited in Appendix A, from which 864 models were selected for inclusion in the main benchmark.

The third and final stage of analysis used the aggregated results from both pipelines and applied statistical analysis to address the research questions. Each comparison was conducted with paired Wilcoxon signed-rank tests corrected by Bonferroni adjustment [32], ensuring robust inference across multiple hypotheses.

The DAGHAR datasets used in this study are already balanced across activity classes. Therefore, standard accuracy is equivalent to balanced accuracy in this setting. For this reason, we adopt accuracy as the primary evaluation metric, as it is the predominant metric used for DAGHAR benchmarks [5], [18], [33].

The statistical tests were performed depending on the research question. For example, to answer what is the best performing encoder overall in SSL we performed a paired Wilcoxon test varying the encoder among all the SSL executions. The same was made for supervised learning and for the other research questions. This rigorous statistical analysis was conducted to support the findings and better classify the top configurations found in this work.

VI. RESULTS AND DISCUSSION

We structured our analysis around the research questions stated in Section I.

A. RQ1: WHAT IS THE BEST-PERFORMING ENCODER OVERALL IN SUPERVISED AND SSL SETTINGS FOR HAR TASKS?

The second stage combined SSL techniques, encoders, refinement strategies, datasets, and labeled fractions, resulting in the final selection of 7,776 experiments, incorporating the 864 selected supervised runs to the 6,912 SSL runs, as detailed in Table 6.

We evaluated six encoder architectures under both supervised and self-supervised (SSL) training methods. In this initial analysis, we did not apply the freeze refinement strategy to ensure a fair comparison with the supervised approach, since full fine-tuning typically results in higher performance and freezing the backbone on the supervised setting would lead to random representations. We examine how freeze refinement affects overall model performance and the relative ranking of the backbones in the next section.

Each of the six encoders was evaluated on six target datasets, using five training methods (four SSL and one supervised) and eight data regimes. This setup resulted in 200 configurations per backbone, each executed three times with different random seeds. To compare results, we applied the Wilcoxon signed-rank test with Bonferroni correction and summarized the outcomes in a precedence graph (Fig. 3). Additionally, we generated a comparative table stratified by training method (Table 7). In the precedence graph, each node represents an encoder, and a directed edge from node A to node B indicates that A significantly outperformed B across repeated runs. Encoders without incoming edges are considered dominant and are shown in green, while those with incoming edges are statistically inferior to others (p-values are indicated on the edges).

Fig. 3 reveals that the ResNet-SE-5 encoder dominates the overall comparison, as evidenced by the absence of incoming edges. Conversely, models such as the IMU Transformer and RNN occupy lower positions in the hierarchy, receiving multiple incoming edges that denote significantly lower performance. Intermediate encoders such as CNN-PFF, TS2Vec, and TS-TCC demonstrate competitive behavior, forming a mid-tier group in the performance hierarchy.

TABLE 7. Performance comparison stratified by training method (supervised + 4 SSL). Best performing encoder for each technique is highlighted in bold and best result overall is colored in gray. The supervised baseline shows ResNet-SE-5 as the best performing encoder. For SSL techniques: ResNet-SE-5 dominates TNC, LFR, and DIET while CNN-PFF performs best with TF-C. Performance shows mean accuracy ± standard deviation. Values in parentheses summarize statistical test results (wins-losses, net advantage).

	Supervised	SSL Technique + Full Fine-Tuning			
Encoder	Baseline	TF-C	TNC	LFR	DIET
ResNet-SE-5	**70.7 ± 12.4 (4-0, +4)**	72.3 ± 16.0 (1-1, 0)	**71.0 ± 11.9 (4-0, +4)**	**71.6 ± 12.2 (3-0, +3)**	**70.0 ± 14.6 (5-0, +5)**
TS2Vec Encoder	66.2 ± 17.7 (1-0, +1)	74.1 ± 15.4 (1-0, +1)	65.6 ± 17.2 (1-0, +1)	65.5 ± 21.0 (0-0, 0)	56.7 ± 24.4 (0-1, -1)
TS-TCC Encoder	66.0 ± 12.8 (1-1, 0)	71.0 ± 12.3 (1-1, 0)	65.0 ± 12.3 (1-1, 0)	68.3 ± 14.3 (1-0, +1)	62.3 ± 16.2 (2-1, 1)
CNN-PFF	63.4 ± 18.8 (1-1, 0)	**76.9 ± 14.6 (3-0, +3)**	63.6 ± 19.4 (1-1, 0)	62.2 ± 18.7 (0-2, -2)	59.7 ± 18.1 (2-1, 1)
IMU Transformer	63.5 ± 10.8 (1-1, 0)	60.8 ± 14.8 (0-5, -5)	63.0 ± 10.7 (1-1, 0)	65.4 ± 13.3 (0-1, -1)	47.8 ± 23.5 (0-3, -3)
RNN	52.7 ± 15.4 (0-5, -5)	71.8 ± 14.9 (1-0, +1)	52.4 ± 14.9 (0-5, -5)	62.3 ± 12.7 (0-2, -2)	52.7 ± 15.5 (0-3, -3)
Average	63.7% ± 15.9%	**71.1% ± 15.5%**	63.4% ± 15.7%	65.9% ± 16.0%	58.2% ± 20.3%

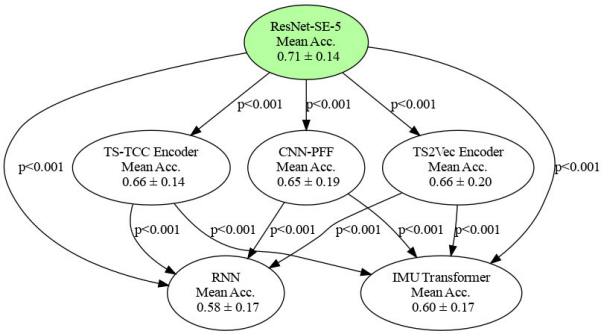


FIGURE 3. Statistical test results grouped by encoder under full fine-tuning. Pairwise comparisons were performed using paired Wilcoxon signed-rank tests with Bonferroni correction to account for multiple hypotheses. ResNet-SE-5 is the best encoder in the overall scenario.

Table 7 presents the numerical results stratified by training method, including mean accuracy, standard deviation, and the number of statistically significant wins and losses, corresponding to the output and input edges in the precedence graphs. Once again, the ResNet-SE-5 backbone achieves the best overall performance, significantly outperforming the others across most training methods. The only exception occurs with the TF-C training technique, where the CNN-PFF backbone surpasses all others, including ResNet-SE-5, achieving an average accuracy of 76.9%, the highest among all configurations.

As discussed in Section IV-B, the TF-C method requires backbones to be integrated with additional network components during downstream evaluation.[2] We attribute the performance differences observed in TF-C to this architectural integration.

These findings underscore the importance of understanding the interplay between encoder architectures and training strategies. Overall, ResNet-SE-5 emerges as the most reliable encoder across both supervised and SSL methods (TNC, LFR, and DIET), while CNN-PFF paired with TF-C achieves the highest absolute accuracy. Among the SSL approaches, LFR generally yields higher performance than TNC, DIET, and the supervised baseline, suggesting it is the most effective of them. However, a direct comparison between LFR and TF-C is not entirely fair, as TF-C incorporates additional architectural components.

In summary, ResNet-SE-5 is the best-performing backbone overall for smartphone-based HAR, while the CNN-PFF-TF-C combination achieves the highest performance in its specific configuration. The next research questions will further dissect these results by data regime, dataset, and refinement strategy, as these factors may also influence performance outcomes.

B. RQ2: HOW IS THE PERFORMANCE OF ENCODERS AFFECTED BY THE FREEZE REFINEMENT STRATEGY?

The choice of fine-tuning strategy influences model performance in HAR, and its impact varies by architecture. Since it would not make sense to freeze a random backbone, we only evaluate refinement strategies under the SSL setting.

Table 8 shows that full fine-tuning leads to higher accuracy, with ResNet-SE-5 + Full Fine-tuning achieving the best average performance (71.2% ± 13.8%), while TS-TCC Encoder, TS2Vec Encoder, and CNN-PFF also perform competitively when fully fine-tuned.

The table also shows that the convolutional encoders (CNN-PFF and TS-TCC) are the overall best performing on the freeze setting. It is also possible to see that the CNN-PFF-TF-C combination is standing out with the highest performances for both refinement strategies, and that the ResNet-SE-5 backbone is the best-performing backbone only under the full fine-tuning refinement strategy.

In summary, the choice of best backbone clearly depends on the refinement strategy. Moreover, the full fine-tuning strategy usually offers the best results and, in this setting, the best overall backbone is the ResNet-SE-5.

C. RQ3: WHAT HAS A GREATER IMPACT ON FINAL PERFORMANCE: CHANGING THE PRE-TRAINING TECHNIQUE OR THE ENCODER ARCHITECTURE?

Knowing that encoders and pre-training method affects the performance of the classifiers for HAR, we investigated which factor impacts more the performance: changing the encoder or changing the pre-training technique. We conducted a comparative analysis based on the results from

[2]The final TF-C architecture is composed of $G_T + G_F + R_T + R_F$, with our backbones replacing G_T and G_F.

TABLE 8. Performance comparison across SSL techniques and refinement strategies. Values show mean accuracy ± standard deviation (%) with Wins-Losses and Net Score in parentheses. Bold indicates best encoder average per strategy, underline indicates second best. Gray indicates best performance per refinement strategy. Freeze refinement strategy substantially reduces performance compared to full fine-tuning. CNN-PFF achieves best frozen overall performance, while ResNet-SE-5 excels only with full fine-tuning.

	Encoders					
SSL Technique	**CNN-PFF**	**ResNet-SE-5**	**TS-TCC**	**TS2Vec**	**IMU Trans.**	**RNN**
Full Finetune						
TF-C	**76.9 ± 14.6 (3-0, +3)**	72.3 ± 16.0 (1-1, 0)	71.0 ± 12.3 (1-1, 0)	74.1 ± 15.4 (1-0, +1)	60.8 ± 14.8 (0-5, -5)	71.8 ± 14.9 (1-0, +1)
TNC	63.6 ± 19.4 (1-1, 0)	71.0 ± 11.9 (4-0, +4)	65.0 ± 12.3 (1-1, 0)	65.6 ± 17.2 (1-0, +1)	63.0 ± 10.7 (1-1, 0)	52.4 ± 14.9 (0-5, -5)
LFR	62.2 ± 18.7 (0-1, -1)	71.6 ± 12.2 (3-0, +3)	68.3 ± 14.3 (1-0, +1)	65.5 ± 21.0 (0-0, 0)	65.4 ± 13.3 (0-1, -1)	62.3 ± 12.7 (0-2, -2)
DIET	59.7 ± 18.1 (2-1, +1)	70.0 ± 14.6 (5-0, +5)	62.3 ± 16.2 (2-1, +1)	56.7 ± 24.4 (0-1, -1)	47.8 ± 23.5 (0-3, -3)	52.7 ± 15.5 (0-3, -3)
Average	65.6 ± 19.0 (2-1, +1)	**71.2** ± 13.8 (5-0, +5)	<u>66.6</u> ± 14.2 (2-1, +1)	65.5 ± 20.7 (2-1, +1)	59.2 ± 17.6 (0-4, -4)	59.8 ± 16.6 (0-4, -4)
Freeze						
TF-C	**69.0 ± 18.9 (4-0, +4)**	60.5 ± 17.7 (1-1, 0)	60.7 ± 14.9 (1-1, 0)	65.1 ± 16.5 (2-0, +2)	49.4 ± 11.9 (0-5, -5)	58.2 ± 14.7 (1-2, -1)
TNC	57.9 ± 18.1 (2-0, +2)	56.4 ± 15.0 (1-1, 0)	62.3 ± 11.3 (3-0, +3)	58.7 ± 14.1 (2-0, +2)	52.9 ± 10.4 (1-3, -2)	42.8 ± 11.8 (0-5, -5)
LFR	60.0 ± 17.8 (1-0, +1)	50.0 ± 15.1 (0-5, -5)	63.4 ± 14.7 (2-0, +2)	57.8 ± 20.6 (1-0, +1)	58.9 ± 15.3 (1-0, +1)	57.6 ± 13.3 (1-1, 0)
DIET	58.6 ± 17.8 (3-0, +3)	58.3 ± 17.0 (3-0, +3)	56.2 ± 15.6 (2-0, +2)	50.7 ± 22.3 (0-2, -2)	44.6 ± 21.9 (0-3, -3)	50.4 ± 13.7 (0-3, -3)
Average	**61.4** ± 18.6 (3-0, +3)	56.3 ± 16.7 (2-2, 0)	<u>60.7</u> ± 14.5 (3-0, +3)	58.1 ± 19.3 (2-0, +2)	51.4 ± 16.4 (0-4, -4)	52.2 ± 14.8 (0-4, -4)

all datasets using the full fine-tuning strategy, which was proved to be the most effective one in the previous research question.

The method used to answer this question was:

1) **Pre-training technique** comparisons: For each dataset and encoder combination, we computed the mean accuracy for each pre-training technique. Then, we calculated the mean pairwise differences between every pair of techniques, measuring the mean performance gain or loss when changing from one technique to another, denoted as μ.
2) **Encoder comparisons:** Similarly, for each dataset and training technique combination, we computed the mean performance of each encoder. Pairwise comparisons were made between encoders to measure the impact of architectural changes.
3) **Result aggregation:** All comparisons were collected and grouped by the type of change (pre-training technique or encoder) and the specific pair being compared. For each comparison, we calculated the average performance difference and standard deviation.
4) **Analysis:** We then sorted the results by average μ and presented the values at Fig. 4a and Fig. 4b.

Fig. 4a shows that changing the training technique results in a mean difference of 5.67 percent points in accuracy, with the biggest differences being observed when changing the usage from DIET to other techniques. Changing from Supervised to TF-C and LFR also yielded improvements in the performance, which suggests that SSL can be a good replacement for supervised learning, if an appropriate technique is chosen.

To check the impact of changing the encoder, a similar analysis was made in Fig. 4b, that shows an average gain of 5.57 percent points. The biggest impact observed was moving from RNN and IMU Transformer to convolutional encoders, confirming the findings of the previous discussions at the first research question.

Hence, we verified that both changing the encoder and the training technique are factors that impact the performance of HAR classifiers, with very close average impact (5.57% vs 5.67%). This result shows that both the encoder and the pre-training technique are important factors for smartphone-based HAR. From the analysis, ResNet-SE-5 emerges as the most consistently effective encoder, while TF-C stands out as the SSL technique with the best performance.

However, it is not sufficient to simply combine the best encoder and the best SSL technique independently (*e.g.*, ResNet-SE-5 and TF-C), as their interaction also affects the final performance. For instance, when focusing solely on TF-C, the CNN-PFF backbone achieves the best results (as shown in Table 7), highlighting the importance of synergy between encoder and training technique.

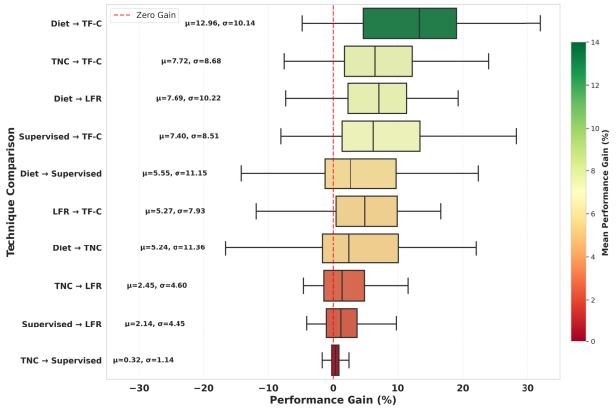
D. RQ4: HOW DOES ENCODER PERFORMANCE VARY ACROSS DIFFERENT TARGET DATASETS?

In this section, we examine whether the optimal encoder choice differs across various target datasets.

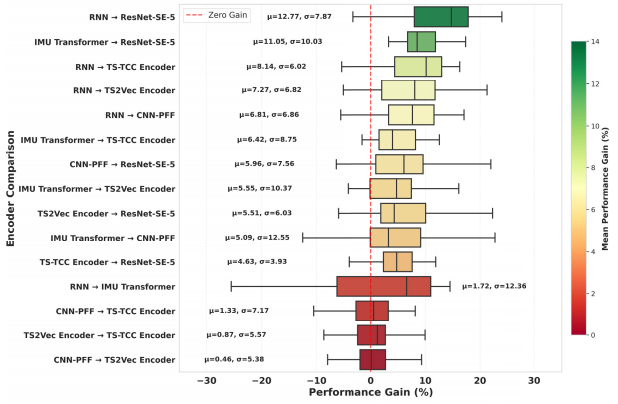
In RQ2, we found that full fine-tuning generally outperforms freezing as a refinement strategy. Accordingly, this subsection focuses primarily on full fine-tuning to address RQ4, while a detailed comparison of both refinement strategies across all datasets is provided in Appendix B.

Table 9 shows how encoder performance varies across datasets. ResNet-SE-5 again stands out as the strongest and most stable encoder, achieving the highest average accuracy in every dataset. Nevertheless, TS-TCC and TS2Vec encoders remain competitive, frequently emerging as reliable second-best options. Overall, these findings suggest that the optimal encoder choice remains consistent across datasets.

The CNN-PFF encoder is once again superior when considering only the experiments with the best SSL technique: TF-C. In this scenario, the CNN-PFF is the best-performing



(a) Distribution of performance gains when changing the training technique, aggregated across all encoder architectures. Positive values indicate improvement when replacing the technique on the left by the one on the right. Results are sorted by mean performance gain (μ). Overall average gain: 5.67 ± 7.82.



(b) Distribution of performance gains when changing the encoder architecture, aggregated across all training techniques. Positive values indicate improvement when replacing the encoder on the left by the one on the right. Results are sorted by mean performance gain (μ). Overall average gain: 5.57 ± 7.82.

FIGURE 4. Impact analysis of changing the pre-training technique or the encoder on HAR classification performance. Green indicates the biggest gains observed. Both factors yield similar average performance gains, with wide variability across combinations. Technique changes can produce high maximum gains in some cases, while encoder replacements, especially from RNN or IMU Transformer to convolutional architectures, tend to provide more consistent improvements.

encoder across almost all datasets, with the exception of UCI, where the TS2Vec encoder achieves slightly better results than the CNN-PFF. These results corroborate the findings of RQ1 and further reinforce that the encoder choice remains consistent across most datasets.

E. RQ5: WHAT IS THE IMPACT OF THE LABELED DATA FRACTION ON THE PERFORMANCE OF DIFFERENT ENCODERS?

In this section, we examine whether the optimal encoder choice changes when training models with different amounts of data. This scenario is particularly relevant for HAR tasks, where labeled data may be scarce and SSL can leverage large volumes of unlabeled data while fine-tuning is performed with only a limited fraction of annotations.

The results in Table 10 show that the amount of labeled data available for refining the model strongly influences performance and can affect which backbone is best suited for the task. When only a few samples per class is available (1-100), ResNet-SE-5 frequently achieves the highest accuracies, demonstrating strong robustness in low-data regimes. As the amount of labeled data increases, however, the TS2Vec encoder scales more effectively, surpassing ResNet-SE-5 and achieving the best overall performance on the largest datasets. This pattern holds for both supervised approaches and most SSL techniques. The main exception is TF-C, for which CNN-PFF consistently dominates. Even in this case, though, the TS2Vec encoder becomes a strong competitor when refining the model with the full dataset (max), making it the preferred choice in high-data scenarios.

The CNN-PFF encoder is often the best choice when combined with TF-C, performing worse than others (TS-TCC and TS2Vec) only in extremely scarce data regimes, such as when training the model with a single sample per class.

Overall, these results show that the amount of labeled data available for refinement strongly influences which backbone is best suited for the task. ResNet-SE-5 is generally the most effective in low-data scenarios, whereas the TS2Vec encoder becomes the preferred option when larger amounts of labeled data are available. Finally, CNN-PFF remains the best choice when training with TF-C, falling behind other encoders only in extremely scarce data regimes.

F. RQ6: WHAT IS THE MINIMUM AMOUNT OF LABELED DATA REQUIRED TO ACHIEVE NEAR-MAXIMUM PERFORMANCE, AND WHAT IS THE GAIN PROVIDED BY USING SSL IN SCARSE DATA SCENARIOS?

To organize this analysis, this subsection is divided into two parts. We first provide an overview of the best and worst performances found across datasets, comparing supervised models with SSL models trained under the full fine-tuning refinement strategy. Then, we present a detailed table reporting the best results per dataset and sample size, highlighting the gains of SSL over supervised learning in each scenario.

1) GENERAL PICTURE

Fig. 5 brings the best and the worst results achieved across all combinations tested, marking with a golden star the maximum accuracy achieved in each dataset. Each point in the chart is marked with the combination of backbone and SSL technique that achieved that result.

As discussed in Section IV-B, the TF-C method contains additional network components during downstream evaluation that can increase performance, hence, we also considered the best SSL performance of the other SSL techniques without considering TF-C in the orange curve. By comparing it to the supervised learning black curve, we can see that in

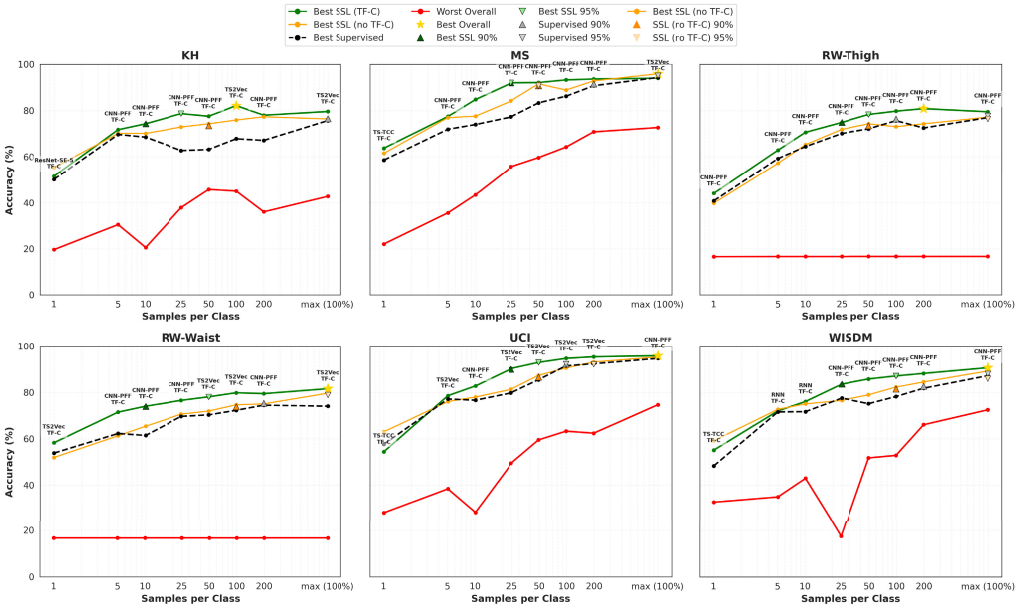


FIGURE 5. Comparison of accuracies between best supervised models and best/worst SSL models across six datasets with all encoders at full fine-tuning strategy. Green lines indicate the best TF-C performance, while orange indicate the best SSL performance excluding TF-C, red shows the overall worst performance, and the black line shows best supervised learning results. Performance is shown for varying samples per class of labeled training data. The encoders used with TF-C are annotated for each scenario. The maximum performance achieved is shown as a golden star and triangles show the point where each curve reaches 90 and 95% of the maximum performance found. It is possible to see that full fine-tuned SSL can be better than supervised learning especially under few-shot learning.

some cases like RW datasets it is similar to supervised, but in others like MS and KH the same pattern of SSL exceeding supervised performance is observed.

To answer what is the minimal amount of labeled data required to achieve a performance close to the best results of each dataset, we marked with triangles the spot where a model was able to achieve 90 or 95% of the best overall accuracy for that dataset marked as a golden star. The markers are placed in each one of the green, orange and black curves.

As we can observe in Fig. 5, in every case of the best TF-C model, using 25 samples per class of the labeled dataset is enough to produce classifiers that reach over 90% of the maximum performance. For other SSL techniques, more samples are needed, and this threshold is achieved when using 100 samples per class. For supervised learning, this happened with even more samples using up to the entire dataset. This highlights the power of SSL pretraining to enhance model training on low-data regimes.

For every dataset, with exception of WISDM, which required 100 samples per class, using TF-C and fine-tuning with 25 to 50 samples produced classifiers that reach over 95% of the maximum performance. This indicates that not only can SSL outperform supervised learning, but it can also do so with a small amount of labeled data, enabling faster and more efficient training for HAR datasets. However, to achieve that data efficiency at the majority of datasets, the TF-C technique was required.

Other aspect shown in Fig. 5 is the worst performance obtained for each dataset and number of samples used at training, represented at the red line. It is notable that performance generally improves as more data is provided and that the gap is big from the worst to the best models assessed in this study. One exception for the rising performances occurs in the RW-Thigh and RW-Waist datasets when using the IMU Transformer paired with DIET, where the encoders failed to learn effective data representations even with the entire dataset for training.

2) TABULAR COMPARISON OF BEST PERFORMANCES

Table 11 shows, based on dataset and samples per class what is the impact of using the supervised learning, SSL with TF-C and SSL with other techniques under full fine-tuning in the best scenarios. The same analysis conducted with the freezing refinement strategy is available at Appendix C.

In supervised learning, ResNet-SE-5 was the most effective encoder in 62.5% of cases. However, they mostly correspond to few-shot learning, and as data increases, other encoders also achieve the maximum performance, such as TS2Vec Encoder and CNN-PFF, each of them in two datasets.

The results change when looking at SSL with full fine-tuning, where we see that the best SSL configurations consistently achieved higher performance than supervised learning, frequently using TF-C.

Full fine-tuning enables multiple SSL techniques to succeed, while TF-C provides the highest absolute performance in most cases (78.7% average), alternative techniques like

TABLE 9. Detailed encoder performance with full fine-tuning strategies for each combination of training strategy and dataset. Results indicate Mean ± Std accuracy (%) and net score in parentheses. Best performer highlighted in bold, second best underlined. Gray background applied only to average row for best and second-best encoders. Average row contains the average of each encoder by training strategy and the sum of the net scores of each net score of each encoder. ResNet-SE-5 achieves the highest and most stable accuracy across datasets. TS2Vec and TS-TCC encoders are the second-best options while CNN-PFF is the leading one on TF-C.

Training Strategy	Dataset	Encoders					
		ResNet-SE-5	CNN-PFF	TS2Vec	TS-TCC	IMU Trans.	RNN
TNC	UCI	**81.7±10.6 (+1)**	74.3±13.4 (+1)	79.0±13.0 (+1)	72.3±13.6 (+1)	71.9±10.8 (+0)	57.7±17.5 (-4)
	RW-Thigh	**62.9±11.3 (+1)**	59.8±16.6 (+0)	60.7±16.6 (+1)	60.4±12.0 (+1)	56.2±8.0 (+0)	46.5±12.9 (-3)
	RW-Waist	64.7±8.0 (+0)	**64.8±12.7 (+1)**	63.0±11.5 (+1)	59.5±10.1 (+0)	58.1±10.1 (+0)	49.5±14.6 (-2)
	MS	**77.5±11.1 (+2)**	69.1±19.6 (+0)	66.3±22.8 (+0)	69.2±10.5 (+0)	66.4±8.9 (-1)	55.0±15.3 (-1)
	WISDM	**74.7±9.1 (+1)**	70.8±12.9 (+0)	71.4±12.4 (+1)	71.5±11.4 (+1)	67.5±8.7 (+0)	60.1±12.8 (-3)
	KH	**64.7±7.0 (+5)**	42.8±22.3 (-1)	53.5±13.5 (-1)	56.8±5.5 (+0)	57.7±7.5 (+0)	45.2±10.4 (-3)
	Average	**71.0±11.9 (+10)**	63.6±19.4 (+1)	65.6±17.2 (+3)	65.0±12.3 (+3)	63.0±10.7 (-1)	52.4±14.9 (-16)
TF-C	UCI	80.6±16.0 (+1)	83.3±15.5 (+1)	**85.3±14.1 (+1)**	76.7±13.2 (+0)	64.4±16.3 (-3)	77.5±19.6 (+0)
	RW-Thigh	66.0±17.8 (+1)	**71.4±12.4 (+1)**	63.4±13.4 (+1)	60.8±12.1 (+1)	48.7±9.5 (-5)	66.3±16.3 (+1)
	RW-Waist	66.5±14.0 (+0)	**72.9±10.1 (+1)**	72.5±8.4 (+1)	67.0±7.8 (+1)	58.7±10.9 (-4)	67.8±9.4 (+1)
	MS	78.0±17.8 (+0)	**83.3±17.7 (+1)**	76.4±21.0 (+0)	75.9±8.2 (+0)	67.9±14.9 (-1)	78.9±13.5 (+0)
	WISDM	73.4±16.2 (+0)	**77.5±16.1 (+0)**	75.2±13.7 (+0)	77.4±11.8 (+0)	69.6±16.4 (+0)	76.6±11.7 (+0)
	KH	69.5±8.4 (+1)	**73.4±10.5 (+1)**	71.5±11.6 (+1)	68.0±11.2 (+1)	55.4±7.7 (-4)	63.6±10.0 (+0)
	Average	72.3±16.0 (+3)	**76.9±14.6 (+5)**	74.1±15.4 (+4)	71.0±12.3 (+3)	60.8±14.8 (-17)	71.8±14.9 (+2)
DIET	UCI	**81.7±12.0 (+2)**	70.2±17.7 (+0)	68.2±27.8 (+0)	69.8±16.4 (+0)	60.7±7.5 (-1)	59.5±19.2 (-1)
	RW-Thigh	**57.2±14.2 (+2)**	47.9±13.9 (+1)	44.9±22.6 (+1)	50.6±13.0 (+1)	16.7±0.0 (-5)	42.2±12.1 (+0)
	RW-Waist	**63.2±12.9 (+2)**	51.2±15.7 (+1)	50.2±21.5 (+1)	52.7±12.3 (+1)	16.7±0.0 (-5)	49.2±10.7 (+0)
	MS	**76.5±12.3 (+2)**	66.9±21.2 (+0)	69.6±26.2 (+0)	70.9±17.1 (+0)	64.2±11. (-1)	58.5±17.5 (-1)
	WISDM	**71.6±12.6 (+2)**	62.6±17.8 (+0)	49.3±24.2 (-3)	70.8±13.9 (+2)	68.3±10.7 (+1)	54.5±14.8 (-2)
	KH	**69.6±9.4 (+4)**	59.4±10.2 (+0)	57.9±11.2 (-1)	59.1±9.5 (-1)	60.2±7.9 (-1)	52.3±11.3 (-1)
	Average	**70.0±14.6 (+14)**	59.7±18.1 (+2)	56.7±24.4 (-2)	62.3±16.2 (+3)	47.8±23.5 (-12)	52.7±15.5 (-5)
LFR	UCI	**81.8±10.2 (+1)**	76.5±12.2 (+0)	77.6±17.5 (+0)	73.6±13.8 (+0)	73.3±10.6 (+0)	67.5±13.0 (-1)
	RW-Thigh	**64.7±12.4 (+1)**	59.8±15.7 (+0)	58.5±19.6 (+0)	61.3±15.3 (+0)	52.6±12.4 (+0)	56.9±13.8 (-1)
	RW-Waist	**65.8±7.4 (+1)**	63.0±12.8 (+0)	61.4±18.0 (+0)	63.5±10.9 (+0)	57.1±10.8 (+0)	57.4±9.6 (-1)
	MS	**79.7±9.2 (+1)**	66.7±20.5 (+0)	71.3±27.2 (+0)	73.2±18.3 (+0)	71.3±11.6 (+0)	66.2±12.6 (-1)
	WISDM	**75.5±10.0 (+0)**	66.1±17.0 (+0)	68.6±18.5 (+0)	72.4±11.8 (+0)	70.5±10.1 (+0)	71.7±9.4 (+0)
	KH	62.2±8.0 (+2)	41.1±13.6 (-5)	55.8±16.7 (+0)	65.7±10.0 (+2)	**67.3±10.0 (+3)**	54.0±6.2 (-2)
	Average	**71.6±12.2 (+6)**	62.2±18.7 (-5)	65.5±21.0 (+0)	68.3±14.3 (+2)	65.4±13.3 (+3)	62.3±12.7 (-6)
Supervised	UCI	**81.9±11.9 (+1)**	75.1±12.2 (+1)	78.8±12.9 (+1)	73.1±13.2 (+0)	72.5±9.8 (+0)	58.0±18.3 (-3)
	RW-Thigh	**63.6±13.4 (+1)**	58.9±19.5 (+0)	62.0±16.2 (+0)	60.1±12.6 (+1)	56.1±9.0 (+0)	47.1±12.7 (-2)
	RW-Waist	**65.3±7.1 (+2)**	64.9±9.9 (+2)	62.5±13.5 (+1)	60.9±11.5 (+0)	58.3±8.6 (-2)	50.0±14.6 (-3)
	MS	**75.7±8.5 (+1)**	67.1±22.0 (+0)	67.8±25.2 (+0)	69.5±11.4 (+0)	69.2±8.9 (+1)	54.9±18.4 (-2)
	WISDM	72.6±12.7 (+1)	69.3±15.2 (+0)	71.7±12.7 (+1)	**72.7±12.5 (+1)**	67.1±9.2 (+0)	59.8±13.8 (-3)
	KH	**65.0±8.9 (+2)**	45.2±17.0 (-1)	54.5±12.6 (+0)	59.6±7.4 (+1)	57.7±7.8 (+1)	46.3±7.6 (-3)
	Average	**70.7±12.4 (+8)**	63.4±18.8 (+2)	66.2±17.7 (+3)	66.0±12.8 (+3)	63.5±10.8 (+0)	52.7±15.4 (-16)

LFR, TNC, and DIET still achieve substantial gains over supervised learning (75.2% average without TF-C).

The results show that optimal performance in full fine-tuned SSL setting is dependent on the technique architecture, as TF-C was used in 90% of the cases as the best SSL technique. In the majority of time TF-C paired with CNN-PFF is the best choice, however TS2Vec is also a competitive encoder choice for this technique.

By looking at the gray cells, where other SSL techniques could achieve a higher performance than TF-C, we can see that this happens mostly at extremely scarse data scenarios, where ResNet-SE-5 excels paired with LFR or TNC. Another case happens at MotionSense, where LFR paired with TS2Vec can achieve the highest performance of the dataset. This shows that while ResNet-SE-5 is good for few-shot learning and can be a general choice, to achieve the best performances other combinations of encoders and SSL techniques are needed.

In summary, full fine-tuning of SSL models consistently outperformed supervised learning across datasets and sampling sizes, being able to achieve the highest accuracy with only a fraction of labeled data. CNN-PFF paired with TF-C emerges again as the most effective and frequent combination, while TS2Vec with TF-C or LFR provides the best results in some datasets, especially when larger sample sizes are available. To achieve high performance on new datasets or problems, it is recommended to prioritize full fine-tuning SSL and consider these combinations of encoders and SSL techniques as starting points, introducing a few adjustments based on dataset size and characteristics.

Therefore, the advantage of SSL is to require fewer labeled samples whilst being able to offer similar of higher performances when compared to supervised approaches.

VII. CONCLUSION AND FUTURE WORK

In this work, we conducted a systematic benchmark evaluating six deep learning encoders across four SSL pretext tasks, two refinement strategies, six HAR datasets, and different

TABLE 10. Performance comparison across SSL techniques with varying labeled data fractions. Best performance per technique and data fraction is highlighted, best performance per data fraction is colored gray and best performance overall is underlined. TF-C dominates across most data regimes, particularly with CNN-PFF encoder. LFR excels at maximum data with TS2Vec encoder (83.8%). ResNet-SE-5 consistently perform well across all techniques, especially in low-data regimes. TF-C shows remarkable data efficiency, achieving over 80% accuracy with just 25 samples using CNN-PFF. DIET performs competitively with ResNet-SE-5 backbones but struggles with other encoders. TNC with ResNet-SE-5 is good at extremely scarse dat scenarios. Overall, SSL techniques significantly outperform supervised baselines, with TF-C providing the most robust performance across data scales.

	Amount of samples per class used to refine the model for the target task							
Technique + Encoder	**1**	**5**	**10**	**25**	**50**	**100**	**200**	**max**
Supervised								
ResNet-SE-5	**50.3 ± 13.4**	**68.7 ± 7.3**	**69.3 ± 6.5**	**71.2 ± 8.2**	**74.5 ± 9.0**	75.5 ± 9.1	76.7 ± 9.6	79.4 ± 9.8
TS2Vec Encoder	36.1 ± 14.2	56.6 ± 11.6	60.1 ± 11.2	69.8 ± 7.7	71.7 ± 9.4	**76.8 ± 9.0**	**77.6 ± 11.7**	**81.0 ± 13.0**
TS-TCC Encoder	45.2 ± 6.8	57.8 ± 8.6	60.0 ± 8.6	69.5 ± 8.2	71.8 ± 6.7	73.3 ± 7.7	74.1 ± 7.0	76.1 ± 11.2
CNN-PFF	34.2 ± 15.7	51.9 ± 12.2	55.4 ± 13.0	68.9 ± 9.4	71.0 ± 13.2	74.6 ± 10.3	73.7 ± 14.8	77.7 ± 12.4
IMU Transformer	47.0 ± 8.8	58.4 ± 7.5	62.9 ± 9.0	66.1 ± 8.0	68.1 ± 7.0	68.3 ± 6.8	66.7 ± 9.5	70.4 ± 9.6
RNN	31.7 ± 7.2	37.7 ± 4.5	40.6 ± 4.5	52.2 ± 7.1	59.9 ± 7.0	64.5 ± 8.7	65.9 ± 9.6	69.1 ± 9.9
TF-C								
ResNet-SE-5	39.6 ± 13.4	66.7 ± 5.2	69.7 ± 8.1	75.7 ± 5.6	79.7 ± 7.1	81.6 ± 8.1	82.1 ± 8.9	83.6 ± 8.4
TS2Vec Encoder	49.6 ± 8.8	63.8 ± 11.4	67.0 ± 11.3	76.9 ± 11.0	80.4 ± 10.1	84.8 ± 8.0	84.0 ± 8.9	86.0 ± 8.9
TS-TCC Encoder	**50.6 ± 12.1**	65.7 ± 6.9	68.5 ± 7.7	73.9 ± 7.5	75.7 ± 7.4	76.4 ± 7.2	77.0 ± 10.3	79.7 ± 10.1
CNN-PFF	45.3 ± 6.9	**70.4 ± 6.1**	**77.0 ± 6.2**	**82.3 ± 6.7**	**83.6 ± 6.6**	**85.1 ± 6.8**	**85.8 ± 7.0**	**86.1 ± 7.7**
IMU Transformer	40.2 ± 10.4	52.3 ± 9.9	55.5 ± 8.5	60.4 ± 8.7	66.4 ± 11.0	67.6 ± 10.8	68.7 ± 12.5	75.3 ± 13.1
RNN	44.9 ± 13.7	61.9 ± 6.8	69.5 ± 6.8	76.1 ± 7.7	78.0 ± 8.4	78.8 ± 9.2	81.4 ± 8.4	83.7 ± 8.6
TNC								
ResNet-SE-5	**53.0 ± 10.2**	**66.9 ± 8.9**	**69.6 ± 6.6**	**71.2 ± 8.2**	**74.2 ± 7.5**	**76.4 ± 8.5**	**78.8 ± 10.6**	78.1 ± 11.8
TS2Vec Encoder	37.5 ± 14.2	55.8 ± 10.9	59.6 ± 10.8	68.8 ± 7.7	71.2 ± 10.3	74.1 ± 9.0	76.7 ± 12.1	**81.3 ± 12.9**
TS-TCC Encoder	45.6 ± 6.0	57.3 ± 7.9	60.7 ± 7.9	67.9 ± 7.7	70.6 ± 6.8	71.7 ± 7.9	71.3 ± 8.5	74.7 ± 12.9
CNN-PFF	35.6 ± 13.4	51.3 ± 12.8	53.8 ± 17.0	72.2 ± 6.5	71.1 ± 15.6	71.8 ± 16.7	75.1 ± 11.0	78.0 ± 13.7
IMU Transformer	46.3 ± 8.4	58.8 ± 6.6	61.3 ± 7.3	65.2 ± 9.0	66.9 ± 7.9	67.1 ± 8.4	68.8 ± 6.8	69.3 ± 9.8
RNN	31.2 ± 8.4	37.7 ± 4.7	42.0 ± 5.0	51.7 ± 6.9	59.6 ± 7.0	63.4 ± 8.5	65.9 ± 9.1	67.3 ± 9.1
LFR								
ResNet-SE-5	**52.4 ± 10.1**	**68.9 ± 9.2**	**70.8 ± 7.1**	**73.0 ± 6.5**	74.8 ± 9.6	75.8 ± 10.5	77.5 ± 10.6	79.8 ± 11.2
TS2Vec Encoder	29.4 ± 13.0	49.1 ± 10.4	55.8 ± 12.8	71.4 ± 9.3	**75.2 ± 14.0**	**78.4 ± 9.6**	**81.1 ± 9.8**	**83.8 ± 10.4**
TS-TCC Encoder	41.2 ± 8.8	56.7 ± 4.8	65.0 ± 6.5	73.9 ± 5.9	75.1 ± 5.2	75.9 ± 6.4	77.2 ± 6.7	81.3 ± 9.6
CNN-PFF	37.9 ± 11.4	49.0 ± 14.2	55.5 ± 12.4	65.7 ± 15.9	72.5 ± 8.3	71.5 ± 13.0	71.6 ± 17.8	74.0 ± 18.5
IMU Transformer	44.4 ± 9.7	58.2 ± 10.8	64.0 ± 10.9	68.6 ± 8.7	70.2 ± 8.7	70.5 ± 9.0	70.6 ± 9.8	76.2 ± 8.8
RNN	41.6 ± 6.5	53.2 ± 9.6	58.7 ± 8.4	64.8 ± 6.4	69.0 ± 8.3	69.2 ± 7.9	70.5 ± 8.7	71.3 ± 9.2
DIET								
ResNet-SE-5	**44.3 ± 10.7**	**62.7 ± 10.3**	**66.9 ± 10.7**	**73.8 ± 6.8**	**76.4 ± 7.6**	**77.1 ± 8.9**	**77.9 ± 10.2**	80.7 ± 9.8
TS2Vec Encoder	32.1 ± 11.1	39.0 ± 18.4	39.9 ± 18.4	49.0 ± 26.4	64.9 ± 16.5	71.4 ± 15.1	75.4 ± 14.0	**82.0 ± 11.4**
TS-TCC Encoder	38.8 ± 5.6	48.4 ± 7.7	54.8 ± 9.3	63.7 ± 11.5	68.6 ± 11.6	70.9 ± 11.4	74.0 ± 10.9	79.2 ± 9.4
CNN-PFF	30.1 ± 7.7	43.9 ± 10.7	52.5 ± 11.5	64.3 ± 9.6	67.3 ± 10.7	69.9 ± 9.9	72.9 ± 9.8	76.6 ± 10.5
IMU Transformer	35.3 ± 14.2	45.4 ± 21.7	47.6 ± 23.3	49.6 ± 24.6	49.7 ± 24.6	51.0 ± 25.4	50.1 ± 24.9	53.6 ± 27.3
RNN	30.4 ± 5.2	38.5 ± 4.3	43.6 ± 5.1	52.5 ± 9.6	57.3 ± 8.8	62.8 ± 8.3	65.8 ± 10.5	70.9 ± 10.5

data availability scenarios. To address six research questions related to encoder selection, SSL effectiveness, refinement strategies, and low-data performance, we trained, in total, 11,232 models, establishing strong supervised baselines and providing an overview of performance trends across encoders, SSL methods, and data regimes.

At RQ1, we identified the best-performing encoder in both supervised and SSL settings. Across all experiments, ResNet-SE-5 consistently emerged as the strongest and most stable encoder overall. In contrast, when paired with the TF-C backbone, which comprises multiple components (FFT, two encoders, and two projectors), CNN-PFF achieved the highest performance, demonstrating that specific encoder-technique combinations can surpass even the strongest overall encoders.

At RQ2, we examined the effect of refinement strategy, determining that full fine-tuning outperformed frozen refinements across all encoders and SSL techniques, indicating that the benefits of SSL can be maximized under this refinement strategy.

At RQ3, we quantified the relative importance of encoder architecture versus SSL technique, discovering that both factors had a similar average impact on performance, demonstrating that HAR performance is multifactorial. This means that selecting the best encoder or the best SSL technique in isolation is insufficient: their interaction determines the final performance.

At RQ4, we investigated how encoder performance varies across datasets. ResNet-SE-5 remained the most reliable encoder across most datasets and training techniques, with TS-TCC and TS2Vec encoders being second-best options. TF-C with CNN-PFF is still consistently the best overall choice considering all datasets.

At RQ5, we evaluated the impact of labeled data availability and found that the data regime strongly influences which encoder is most effective. In general, ResNet-SE-5 dominates under scarce labeled data, while TS2Vec becomes the preferred option as the amount of labeled data increases. In the TF-C setting, the CNN-PFF is often the best choice as the backbone encoder; however, TS-TCC

TABLE 11. Comparison of supervised learning versus self-supervised learning (SSL) with full fine-tuning (FT), including ablation without TF-C. Results show the best performing methods for supervised baseline, SSL with full fine-tuning without TF-C, and SSL with full fine-tuning including TF-C. Performance impacts are calculated as the accuracy difference between SSL methods and the supervised baseline. Positive impacts ≥+1% are highlighted in green, negative impacts ≤-1% in red. The summary row shows that full fine-tuning with TF-C achieves the highest overall accuracy (78.7%), outperforming SSL without TF-C (75.2%) and supervised learning (72.5%). Bold marks the highest score within each dataset, whereas underline marks the highest score within each column per dataset. The points where SSL without TF-C achieves a higher performances than TF-C are marked as gray.

Dataset	Samples	Best Supervised	Best SSL (Full FT no TF-C)	Impact	Best SSL (Full FT with TF-C)	Impact
KH	1	50.2±5.0 (ResNet-SE-5)	55.3±3.8 (ResNet-SE-5, TNC)	+5.1	51.6±6.3 (ResNet-SE-5, TF-C)	+1.4
KH	5	69.7±0.4 (ResNet-SE-5)	70.1±2.1 (ResNet-SE-5, TNC)	+0.5	71.8±3.6 (CNN-PFF, TF-C)	+2.1
KH	10	68.5±6.1 (ResNet-SE-5)	70.1±8.5 (IMU Transformer, LFR)	+1.6	74.3±6.1 (CNN-PFF, TF-C)	+5.8
KH	25	62.7±2.0 (TS-TCC Encoder)	72.9±4.2 (TS-TCC Encoder, LFR)	+10.2	78.7±3.2 (CNN-PFF, TF-C)	+16.0
KH	50	63.2±7.9 (ResNet-SE-5)	74.3±2.5 (TS-TCC Encoder, LFR)	+11.1	77.5±1.7 (CNN-PFF, TF-C)	+14.4
KH	100	67.8±7.0 (ResNet-SE-5)	75.9±1.1 (ResNet-SE-5, DIET)	+8.1	**82.2±6.7 (TS2Vec Encoder, TF-C)**	+14.4
KH	200	67.1±7.6 (ResNet-SE-5)	77.3±2.8 (ResNet-SE-5, DIET)	+10.2	78.0±0.8 (CNN-PFF, TF-C)	+10.9
KH	max	75.7±0.7 (ResNet-SE-5)	76.4±1.4 (ResNet-SE-5, DIET)	+0.7	79.6±9.6 (TS2Vec Encoder, TF-C)	+3.9
MS	1	58.6±5.2 (ResNet-SE-5)	61.5±9.5 (ResNet-SE-5, LFR)	+2.9	63.7±8.2 (TS-TCC Encoder, TF-C)	+5.1
MS	5	71.9±2.2 (ResNet-SE-5)	77.0±6.7 (ResNet-SE-5, LFR)	+5.0	77.5±4.7 (CNN-PFF, TF-C)	+5.5
MS	10	74.0±2.5 (ResNet-SE-5)	77.6±0.6 (ResNet-SE-5, LFR)	+3.6	84.8±2.9 (CNN-PFF, TF-C)	+10.8
MS	25	77.3±6.6 (ResNet-SE-5)	84.2±2.6 (TS2Vec Encoder, LFR)	+6.9	92.0±0.5 (CNN-PFF, TF-C)	+14.7
MS	50	83.3±0.7 (ResNet-SE-5)	91.6±0.8 (TS2Vec Encoder, LFR)	+8.2	92.1±1.4 (CNN-PFF, TF-C)	+8.8
MS	100	86.2±1.7 (TS2Vec Encoder)	88.9±3.3 (TS2Vec Encoder, LFR)	+2.7	93.3±0.9 (CNN-PFF, TF-C)	+7.0
MS	200	90.7±3.6 (TS2Vec Encoder)	92.8±1.3 (TS2Vec Encoder, DIET)	+2.1	93.6±1.3 (CNN-PFF, TF-C)	+2.9
MS	max	94.3±1.9 (TS2Vec Encoder)	**95.9±1.8 (TS2Vec Encoder, LFR)**	+1.6	94.0±3.6 (TS2Vec Encoder, TF-C)	-0.3
RW-Thigh	1	40.9±10.7 (IMU Transformer)	39.9±7.1 (IMU Transformer, TNC)	-1.0	44.2±7.3 (CNN-PFF, TF-C)	+3.2
RW-Thigh	5	59.2±4.9 (ResNet-SE-5)	57.2±9.7 (ResNet-SE-5, LFR)	-2.0	62.9±5.6 (CNN-PFF, TF-C)	+3.6
RW-Thigh	10	64.5±2.6 (ResNet-SE-5)	65.3±2.2 (ResNet-SE-5, TNC)	+0.8	70.6±6.0 (CNN-PFF, TF-C)	+6.2
RW-Thigh	25	70.1±1.3 (TS2Vec Encoder)	71.9±3.1 (ResNet-SE-5, LFR)	+1.8	74.9±3.9 (CNN-PFF, TF-C)	+4.8
RW-Thigh	50	72.1±0.8 (CNN-PFF)	74.3±0.4 (ResNet-SE-5, LFR)	+2.2	78.3±1.7 (CNN-PFF, TF-C)	+6.2
RW-Thigh	100	75.6±1.7 (TS2Vec Encoder)	73.1±2.2 (CNN-PFF, TNC)	-2.6	79.8±2.1 (CNN-PFF, TF-C)	+4.1
RW-Thigh	200	72.5±1.8 (CNN-PFF)	74.3±1.9 (CNN-PFF, TNC)	+1.8	**80.9±0.9 (CNN-PFF, TF-C)**	+8.3
RW-Thigh	max	77.0±0.2 (TS2Vec Encoder)	77.2±1.4 (TS2Vec Encoder, TNC)	+0.2	79.5±1.6 (CNN-PFF, TF-C)	+2.5
RW-Waist	1	53.8±5.2 (ResNet-SE-5)	51.9±5.1 (ResNet-SE-5, LFR)	-1.9	58.3±4.8 (TS2Vec Encoder, TF-C)	+4.5
RW-Waist	5	62.3±9.4 (ResNet-SE-5)	61.3±7.5 (ResNet-SE-5, LFR)	-1.0	71.5±1.9 (CNN-PFF, TF-C)	+9.2
RW-Waist	10	61.4±3.8 (CNN-PFF)	65.4±3.9 (ResNet-SE-5, DIET)	+4.0	74.0±1.9 (CNN-PFF, TF-C)	+12.6
RW-Waist	25	69.7±3.8 (ResNet-SE-5)	70.6±3.3 (ResNet-SE-5, LFR)	+1.0	76.6±1.9 (CNN-PFF, TF-C)	+6.9
RW-Waist	50	70.3±2.7 (CNN-PFF)	72.0±2.5 (TS2Vec Encoder, LFR)	+1.6	78.1±3.3 (TS2Vec Encoder, TF-C)	+7.8
RW-Waist	100	72.3±3.9 (TS2Vec Encoder)	74.7±3.6 (TS2Vec Encoder, LFR)	+2.4	79.9±0.8 (TS2Vec Encoder, TF-C)	+7.6
RW-Waist	200	74.5±1.4 (CNN-PFF)	75.1±1.9 (TS2Vec Encoder, LFR)	+0.6	79.5±2.0 (CNN-PFF, TF-C)	+5.0
RW-Waist	max	74.1±5.2 (TS2Vec Encoder)	79.7±1.7 (TS2Vec Encoder, TNC)	+5.6	**81.6±0.9 (TS2Vec Encoder, TF-C)**	+7.5
UCI	1	57.6±9.3 (ResNet-SE-5)	63.0±5.8 (ResNet-SE-5, LFR)	+5.4	54.4±10.6 (TS-TCC Encoder, TF-C)	-3.2
UCI	5	77.1±2.3 (ResNet-SE-5)	76.1±3.9 (ResNet-SE-5, TNC)	-1.1	78.6±3.1 (TS2Vec Encoder, TF-C)	+1.4
UCI	10	76.7±0.9 (ResNet-SE-5)	78.0±0.9 (ResNet-SE-5, TNC)	+1.4	82.8±2.8 (CNN-PFF, TF-C)	+6.1
UCI	25	79.9±1.6 (ResNet-SE-5)	81.4±1.1 (ResNet-SE-5, DIET)	+1.5	90.3±1.5 (TS2Vec Encoder, TF-C)	+10.5
UCI	50	85.5±2.1 (ResNet-SE-5)	87.4±0.9 (ResNet-SE-5, DIET)	+1.9	93.0±0.7 (TS2Vec Encoder, TF-C)	+7.5
UCI	100	91.5±1.6 (ResNet-SE-5)	90.6±1.0 (TS2Vec Encoder, DIET)	-0.9	94.7±0.7 (TS2Vec Encoder, TF-C)	+3.2
UCI	200	92.4±2.0 (TS2Vec Encoder)	93.2±1.4 (ResNet-SE-5, TNC)	+0.8	95.4±0.6 (TS2Vec Encoder, TF-C)	+3.0
UCI	max	94.7±0.7 (ResNet-SE-5)	95.4±0.9 (ResNet-SE-5, DIET)	+0.6	**95.9±0.3 (CNN-PFF, TF-C)**	+1.2
WISDM	1	48.3±12.2 (IMU Transformer)	59.2±13.3 (ResNet-SE-5, TNC)	+10.9	55.1±9.3 (TS-TCC Encoder, TF-C)	+6.8
WISDM	5	71.6±1.6 (ResNet-SE-5)	72.7±5.9 (ResNet-SE-5, LFR)	+1.1	72.0±3.8 (RNN, TF-C)	+0.4
WISDM	10	71.7±1.5 (IMU Transformer)	75.1±1.8 (ResNet-SE-5, LFR)	+3.4	76.1±1.4 (RNN, TF-C)	+4.4
WISDM	25	77.6±0.6 (TS-TCC Encoder)	76.6±0.4 (TS-TCC Encoder, LFR)	-1.0	83.6±0.7 (CNN-PFF, TF-C)	+6.0
WISDM	50	75.2±2.9 (TS-TCC Encoder)	79.0±1.9 (ResNet-SE-5, LFR)	+3.8	85.8±2.0 (CNN-PFF, TF-C)	+10.7
WISDM	100	78.2±1.1 (CNN-PFF)	82.3±2.4 (ResNet-SE-5, LFR)	+4.1	87.1±0.3 (CNN-PFF, TF-C)	+8.9
WISDM	200	81.8±1.6 (ResNet-SE-5)	84.5±0.9 (TS2Vec Encoder, LFR)	+2.7	88.2±1.5 (CNN-PFF, TF-C)	+6.4
WISDM	max	87.3±0.9 (CNN-PFF)	89.2±0.3 (TS2Vec Encoder, LFR)	+2.0	**90.7±0.5 (CNN-PFF, TF-C)**	+3.4
Summary		72.5±3.4	75.2±3.2	**≥+1:33** **≤-1: 7**	**78.7±3.1**	**≥+1:45** **≤-1: 1**

achieved superior results in the extremely scarce labeled-data regime.

At RQ6, we assessed the minimum amount of labeled data required to reach near-maximum performance and the gain provided by SSL. SSL models, especially those trained with TF-C, achieved over 95% of the best accuracy on most datasets with only up to 50 labeled samples per class. Moreover, in extremely scarce data settings, LFR or TNC combined with ResNet-SE-5 can outperform TF-C, and LFR + TS2Vec achieves the highest performance on MotionSense. These exceptions highlight that alternative SSL techniques may offer advantages under specific constraints.

Putting these findings together, our results suggest full-finetuning as the preferred refinement strategy, ResNet-SE-5 as a strong default encoder, and CNN-PFF combined

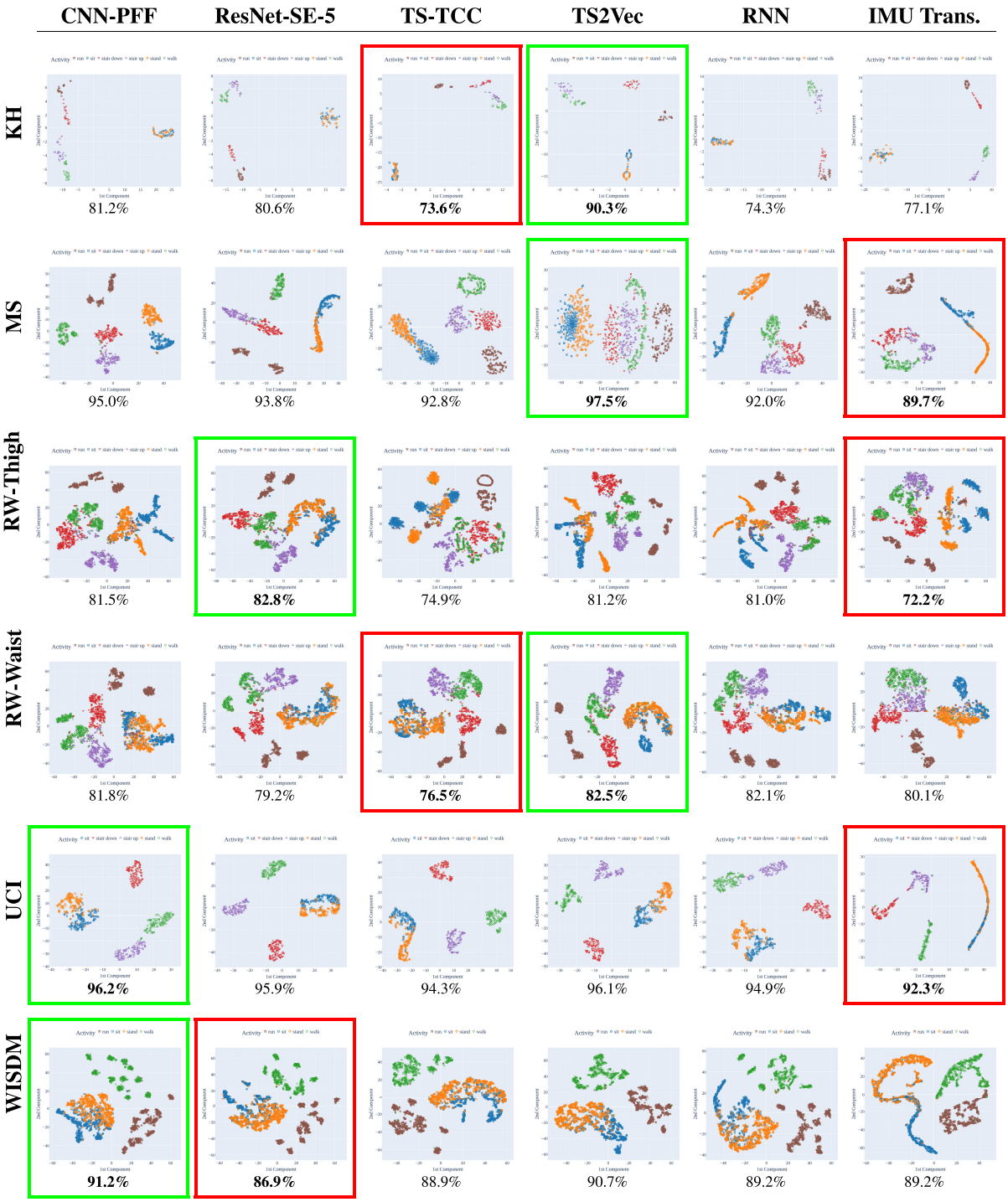


FIGURE 6. T-SNE visualizations of learned representations are shown for six datasets, using the best model identified for each dataset-encoder pair. The highest-performing model for each dataset is highlighted with a green border, while the lowest-performing model is marked with a red border. Accuracy scores are reported below each plot.

with TF-C as an effective SSL configuration, especially in few-shot learning settings.

Finally, this benchmark is designed to be extensible, allowing the inclusion of different encoders, such as DeepSense [16], SW-T [34], and iSPLInception [15]. For transformers, using additional refinement strategies such as LORA [35] may yield more expressive gains for the encoder. As we showed that encoder performance is dataset-dependent, future work can include evaluations on datasets beyond HAR, like fall detection datasets such as UmaFall [36], or even different 1-D signals datasets for Gait Recognition, EEG or other applications. Additional SSL

techniques could also be explored, ranging from classical approaches such as CPC [37] and SimCLR [38] to more recent approaches such as REBAR [10], FOCAL [7], and aLLM4TS [39]. Other metrics may be incorporated to evaluate the performance of encoders, such as F_1-score and training time, as some encoders such as TS2Vec Encoder or techniques such as TNC tend to exhibit substantially higher training cost.

APPENDIX A
LEARNING RATE ABLATION

In order to establish a supervised baseline for each encoder architecture, we performed a learning rate ablation search study, evaluating each encoder across five learning rates (from 10^{-1} to 10^{-4}) using a standard supervised learning training setup. In total, 4,320 supervised models were trained for the ablation study, with the varying fields summarized in Table 5 of Section V. Each configuration was repeated three times with different seeds. The best overall 864 models based on learning rate were selected to be included in the final benchmark, which comprised 7,776 experiments covering various SSL pretext tasks, encoders, datasets, sample sizes, and refinement strategies shown in Table 5.

The results summarized in Table 12 show that performance is highly sensitive to the learning rate, with an optimal value existing for each encoder. Results show that using a learning rate of 10^{-4} was optimal for the majority of the encoders, while TS2Vec Encoder performed best with 10^{-3}. Based on that, we selected the models that used a learning rate of 10^{-4} for all encoders with exception to the TS2Vec Encoder. The "W/L" and "Net" indicated in parenthesis which count wins/losses against other LRs and the net difference confirm the superiority of the chosen learning rates.

The best performing learning rate for each encoder was adopted for all experiments with that encoder, whether in pure supervised learning or as the downstream fine-tuning stage of SSL. This controlled approach ensures that any performance gains demonstrated by our SSL methods are a direct result of the pre-training and not an artifact of better-tuned hyperparameters.

APPENDIX B
DATASET-SPECIFIC ENCODER PERFORMANCE OF SSL UNDER FREEZE (RQ4)

To evaluate the impact of different encoders on different datasets, we bring the complete tabular results for RQ4 based on the freeze refinement strategies.

Table 13 shows that under freeze performance drops relative to Full Fine-tuning and no single encoder dominates across datasets. Instead, encoders different than ResNet-SE-5 emerge depending on the dataset: TS2Vec Encoder achieves the best performance in UCI, CNN-PFF leads in RW-Waist and RW-Thigh, and TS-TCC Encoder ranks highest in MotionSense, WISDM, and KuHAR. This variability confirms that the interaction between dataset properties, encoder design, and refinement strategy is critical to overall performance.

TABLE 12. Ablation study of learning rates across different encoders. Each row reports mean ± std (%) accuracy with W/L and net score. Best learning rate per encoder is shown in bold.

Encoder	Learning Rate	Accuracy
CNN-PFF	10^{-5}	46.3 ± 23.2 (1/2, -1)
	10^{-4}	**63.4 ± 18.8 (3/0, +3)**
	10^{-3}	62.5 ± 19.6 (3/0, +3)
	10^{-2}	51.5 ± 21.7 (1/2, -1)
	10^{-1}	35.6 ± 11.4 (0/4, -4)
ResNet-SE-5	10^{-5}	59.2 ± 16.8 (0/2, -2)
	10^{-4}	**70.7 ± 12.4 (3/0, +3)**
	10^{-3}	69.5 ± 12.8 (3/0, +3)
	10^{-2}	60.3 ± 23.4 (0/2, -2)
	10^{-1}	63.2 ± 12.5 (0/2, -2)
TS-TCC Encoder	10^{-5}	63.8 ± 12.8 (2/0, +2)
	10^{-4}	**66.0 ± 12.8 (2/0, +2)**
	10^{-3}	64.4 ± 14.6 (2/0, +2)
	10^{-2}	56.6 ± 17.7 (1/3, -2)
	10^{-1}	30.6 ± 6.8 (0/4, -4)
IMU Transformer	10^{-5}	55.1 ± 14.1 (2/2, 0)
	10^{-4}	**63.5 ± 10.8 (3/0, +3)**
	10^{-3}	60.7 ± 11.7 (3/0, +3)
	10^{-2}	28.6 ± 9.8 (1/3, -2)
	10^{-1}	18.6 ± 3.1 (0/4, -4)
RNN	10^{-5}	36.8 ± 15.9 (1/3, -2)
	10^{-4}	**52.7 ± 15.4 (2/0, +2)**
	10^{-3}	52.6 ± 17.3 (2/0, +2)
	10^{-2}	51.2 ± 18.7 (2/0, +2)
	10^{-1}	19.5 ± 4.0 (0/4, -4)
TS2Vec Encoder	10^{-5}	18.4 ± 4.8 (0/3, -3)
	10^{-4}	63.8 ± 16.6 (3/0, +3)
	10^{-3}	**66.2 ± 17.7 (3/0, +3)**
	10^{-2}	31.9 ± 16.3 (2/2, 0)
	10^{-1}	18.3 ± 4.3 (0/3, -3)

TABLE 13. Detailed encoder performance with Freeze strategy. For each SSL technique, individual dataset results are shown with average performance below. Mean ± Std accuracy (%) and net score in parentheses. Best performer highlighted in bold, second best underlined. Gray background applied only to average row for best and second best encoders. Average row contains the average of each encoder by training strategy and the sum of the net scores of each net score of each encoder.

Training Strategy	Dataset	Encoders: ResNet-SE-5	CNN-PFF	TS2Vec	TS-TCC	IMU Trans.	RNN
TNC	UCI	62.8±14.1 (+1)	65.1±18.6 (+1)	**72.1±8.6 (+2)**	64.8±10.6 (+2)	52.5±10.3 (-1)	41.0±8.9 (-5)
	RW-Thigh	48.9±12.6 (+1)	53.5±17.0 (+1)	52.1±12.0 (+1)	**54.9±9.9 (+1)**	48.8±9.3 (+1)	37.3±9.6 (-5)
	RW-Waist	51.4±14.2 (+1)	**60.5±13.4 (+2)**	56.9±10.8 (+2)	54.5±8.3 (+2)	47.8±8.1 (-2)	38.0±8.9 (-5)
	MS	59.7±17.7 (+0)	62.9±18.5 (+1)	59.4±15.7 (+0)	**67.4±11.6 (+2)**	54.2±6.7 (-1)	44.9±14.3 (-2)
	WISDM	66.7±9.5 (+1)	66.1±9.7 (+1)	63.3±8.9 (+0)	**69.6±10.6 (+1)**	63.3±9.2 (+0)	53.3±11.6 (-3)
	KH	48.9±11.7 (-1)	39.1±15.18 (-1)	48.3±14.5 (-1)	**62.8±7.9 (+5)**	50.6±10.7 (-1)	42.3±10.2 (-1)
	Average	56.4±15.0 (+3)	57.9±18.1 (+5)	58.7±14.1 (+4)	**62.3±11.3 (+13)**	52.9±10.4 (-4)	42.8±11.8 (-21)
TF-C	UCI	64.4±17.6 (+1)	74.4±19.9 (+2)	**76.6±15.8 (+3)**	58.8±8.9 (+0)	40.9±2.6 (-5)	53.6±10.2 (-1)
	RW-Thigh	56.5±19.7 (+0)	**63.1±18.0 (+1)**	55.7±17.2 (+0)	54.5±12.7 (+1)	43.5±10.6 (-3)	57.7±17.1 (+1)
	RW-Waist	55.6±17.6 (+0)	**64.3±16.8 (+1)**	64.1±15.7 (+1)	54.3±15.2 (+0)	48.7±13.1 (-2)	56.3±9.4 (+0)
	MS	62.1±19.2 (-1)	**76.2±20.7 (+2)**	64.5±19.2 (+0)	71.4±13.2 (+1)	52.9±7.1 (-3)	62.8±14.4 (+1)
	WISDM	63.2±19.2 (+0)	**70.3±21.5 (+0)**	69.8±12.3 (+0)	67.2±17.7 (+0)	59.4±16.7 (+0)	67.5±20.4 (+0)
	KH	61.0±11.8 (+1)	**65.8±13.0 (+2)**	60.1±10.3 (+1)	58.3±12.9 (+0)	51.1±5.8 (-2)	51.2±6.8 (-2)
	Average	60.5±17.7 (+1)	**69.0±18.9 (+8)**	65.1±16.5 (+5)	60.7±14.9 (+2)	49.4±11.9 (-15)	58.2±14.7 (-1)
DIET	UCI	62.6±15.7 (+0)	67.8±17.6 (+0)	**73.6±12.0 (+2)**	61.5±15.8 (+0)	54.3±7.0 (-1)	53.4±15.3 (-1)
	RW-Thigh	**50.5±17.8 (+2)**	45.5±13.6 (+1)	34.4±13.5 (+0)	43.9±10.7 (+1)	16.7±0.0 (-5)	43.4±10.9 (+1)
	RW-Waist	**56.4±18.8 (+2)**	52.1±16.0 (+2)	38.8±11.0 (-2)	46.3±11.0 (+1)	16.7±0.0 (-5)	47.8±9.6 (+2)
	MS	62.9±16.4 (+0)	66.8±19.6 (+0)	**67.2±15.0 (+0)**	63.0±17.3 (+0)	58.6±10.5 (+0)	53.8±17.1 (+0)
	WISDM	61.6±17.5 (+1)	60.5±17.4 (+1)	26.0±16.2 (-5)	**65.9±12.5 (+1)**	64.9±13.7 (+1)	53.0±13.7 (+1)
	KH	56.2±13.7 (+0)	59.2±11.9 (+0)	**64.1±9.5 (+1)**	56.5±11.8 (+0)	56.3±11.7 (+0)	50.8±12.7 (-1)
	Average	58.3±17.0 (+5)	**58.6±17.8 (+4)**	50.7±22.3 (-4)	56.2±15.6 (+3)	44.6±21.9 (-10)	50.4±13.7 (+2)
LFR	UCI	56.6±13.8 (-1)	70.5±15.7 (+1)	**70.6±16.4 (+0)**	68.9±14.9 (+0)	63.8±14.2 (+0)	62.2±11.2 (+0)
	RW-Thigh	41.2±12.8 (-2)	54.5±18.3 (+1)	50.2±16.4 (+0)	**55.0±13.3 (+1)**	47.2±15.4 (+0)	49.5±13.4 (+0)
	RW-Waist	46.0±13.2 (-2)	**61.0±13.1 (+1)**	54.6±16.6 (+0)	58.5±11.7 (+1)	52.3±13.1 (+0)	53.4±12.1 (+0)
	MS	52.8±19.2 (+0)	62.5±22.9 (+0)	60.2±26.1 (+0)	**67.5±18.0 (+0)**	65.3±17.6 (+0)	61.2±14.8 (+0)
	WISDM	53.9±15.0 (-2)	64.0±14.6 (+0)	64.1±18.9 (+0)	**66.8±13.0 (+1)**	65.3±12.7 (+1)	63.3±14.8 (+0)
	KH	49.3±11.7 (-1)	47.7±12.6 (-2)	47.2±19.2 (-1)	**64.0±12.5 (+3)**	59.5±8.8 (+1)	56.0±6.8 (+0)
	Average	50.0±15.1 (-8)	60.0±17.8 (+1)	57.8±20.6 (-1)	**63.4±14.7 (+6)**	58.9±15.3 (+2)	57.6±13.3 (+0)
Supervised		-	-	-	-	-	-

APPENDIX C
TABULAR COMPARISON OF BEST PERFORMANCES UNDER FREEZE (RQ6)

At RQ6, we focused at the full fine-tuning refinement strategy, but freezing is also a relevant scenario for analysis.

Our analysis of the freezing refinement strategy at Table 14 reveals distinct patterns compared to full fine-tuning. Supervised learning showed to have the best performance at five out of six datasets. While frozen SSL with TF-C achieves overall performance comparable to supervised learning (71.1% vs 72.5%), the effectiveness varies significantly across data regimes. Without TF-C, frozen SSL struggles, with only 10% of configurations showing positive impacts $\geq +1\%$, increasing to 38% when TF-C is included.

TABLE 14. Comparison of supervised learning versus self-supervised learning (SSL) with frozen encoders, including ablation without TF-C. Results show the best performing methods for supervised baseline, SSL with frozen encoders without TF-C, and SSL with frozen encoders including TF-C. Performance impacts are calculated as the accuracy difference between SSL methods and the supervised baseline. Positive impacts ≥+1% are highlighted in green, negative impacts ≤-1% in red. The summary row shows that frozen SSL with TF-C achieves comparable overall accuracy (71.1%) to supervised learning (72.5%), while SSL without TF-C performs worse (68.2%). Bold marks the highest score within each dataset, whereas underline marks the highest score within each column per dataset. The points where SSL without TF-C achieves a higher performances than TF-C are marked as gray.

Dataset	Samples	Best Supervised	Best SSL (Freeze no TF-C)	Impact	Best SSL (Freeze with TF-C)	Impact
KH	1	50.2±5.0 (ResNet-SE-5)	47.0±1.7 (TS-TCC Encoder, TNC)	-3.2	41.0±10.9 (IMU Transformer, TF-C)	-9.3
KH	5	69.7±0.4 (ResNet-SE-5)	60.9±6.5 (TS2Vec Encoder, DIET)	-8.8	60.4±2.5 (TS2Vec Encoder, TF-C)	-9.3
KH	10	68.5±6.1 (ResNet-SE-5)	63.4±8.3 (TS2Vec Encoder, DIET)	-5.1	61.1±7.2 (TS2Vec Encoder, TF-C)	-7.4
KH	25	62.7±2.0 (TS-TCC Encoder)	69.2±1.4 (TS-TCC Encoder, LFR)	+6.5	73.4±2.1 (CNN-PFF, TF-C)	+10.6
KH	50	63.2±7.9 (ResNet-SE-5)	71.3±2.4 (TS-TCC Encoder, LFR)	+8.1	74.5±2.9 (CNN-PFF, TF-C)	+11.3
KH	100	67.8±7.0 (ResNet-SE-5)	72.9±2.1 (TS-TCC Encoder, LFR)	+5.1	73.6±3.5 (CNN-PFF, TF-C)	+5.8
KH	200	67.1±7.6 (ResNet-SE-5)	75.2±4.0 (TS-TCC Encoder, LFR)	+8.1	75.2±3.1 (CNN-PFF, TF-C)	+8.1
KH	max	**75.7±0.7 (ResNet-SE-5)**	74.8±1.4 (TS-TCC Encoder, LFR)	-0.9	73.8±4.0 (CNN-PFF, TF-C)	-1.9
MS	1	58.6±5.2 (ResNet-SE-5)	46.5±8.5 (ResNet-SE-5, TNC)	-12.1	43.0±10.7 (TS-TCC Encoder, TF-C)	-15.6
MS	5	71.9±2.2 (ResNet-SE-5)	61.6±2.7 (TS-TCC Encoder, TNC)	-10.3	67.6±2.0 (CNN-PFF, TF-C)	-4.3
MS	10	74.0±2.5 (ResNet-SE-5)	63.2±2.2 (CNN-PFF, DIET)	-10.8	74.7±6.0 (CNN-PFF, TF-C)	+0.7
MS	25	77.3±6.6 (ResNet-SE-5)	73.6±4.6 (CNN-PFF, DIET)	-3.7	86.2±1.1 (CNN-PFF, TF-C)	+8.9
MS	50	83.3±0.7 (ResNet-SE-5)	77.1±5.4 (CNN-PFF, DIET)	-6.3	86.9±1.1 (CNN-PFF, TF-C)	+3.6
MS	100	86.2±1.7 (TS2Vec Encoder)	81.4±5.0 (TS-TCC Encoder, LFR)	-4.8	88.7±0.8 (CNN-PFF, TF-C)	+2.4
MS	200	90.7±3.6 (TS2Vec Encoder)	86.3±1.2 (TS-TCC Encoder, LFR)	-4.4	88.8±1.8 (CNN-PFF, TF-C)	-1.9
MS	max	**94.3±1.9 (TS2Vec Encoder)**	87.1±0.7 (TS-TCC Encoder, LFR)	-7.2	89.7±1.6 (CNN-PFF, TF-C)	-4.6
RW-Thigh	1	40.9±10.7 (IMU Transformer)	40.0±6.3 (TS-TCC Encoder, TNC)	-0.9	30.0±1.1 (TS-TCC Encoder, TF-C)	-10.9
RW-Thigh	5	59.2±4.9 (ResNet-SE-5)	46.0±2.8 (TS-TCC Encoder, TNC)	-13.3	52.0±7.3 (CNN-PFF, TF-C)	-7.2
RW-Thigh	10	64.5±2.6 (ResNet-SE-5)	50.2±2.5 (TS2Vec Encoder, TNC)	-14.2	60.6±11.5 (CNN-PFF, TF-C)	-3.8
RW-Thigh	25	70.1±1.3 (TS2Vec Encoder)	63.2±3.6 (CNN-PFF, LFR)	-6.9	70.6±3.1 (CNN-PFF, TF-C)	+0.4
RW-Thigh	50	72.1±0.8 (CNN-PFF)	67.5±3.9 (CNN-PFF, LFR)	-4.6	68.2±3.4 (ResNet-SE-5, TF-C)	-3.9
RW-Thigh	100	75.6±1.7 (TS2Vec Encoder)	69.0±1.7 (CNN-PFF, LFR)	-6.7	76.4±0.9 (CNN-PFF, TF-C)	+0.8
RW-Thigh	200	72.5±1.8 (CNN-PFF)	68.7±4.4 (CNN-PFF, TNC)	-3.8	76.9±1.3 (CNN-PFF, TF-C)	+4.4
RW-Thigh	max	**77.0±0.2 (TS2Vec Encoder)**	72.8±1.8 (CNN-PFF, TNC)	-4.2	75.7±0.6 (CNN-PFF, TF-C)	-1.3
RW-Waist	1	53.8±5.2 (ResNet-SE-5)	43.6±3.9 (TS-TCC Encoder, TNC)	-10.2	37.9±3.1 (RNN, TF-C)	-15.9
RW-Waist	5	62.3±9.4 (ResNet-SE-5)	50.5±4.5 (TS2Vec Encoder, TNC)	-11.8	56.3±6.5 (CNN-PFF, TF-C)	-6.0
RW-Waist	10	61.4±3.8 (CNN-PFF)	51.6±2.5 (TS-TCC Encoder, LFR)	-9.9	58.6±1.1 (TS2Vec Encoder, TF-C)	-2.9
RW-Waist	25	69.7±3.8 (ResNet-SE-5)	67.1±1.0 (CNN-PFF, LFR)	-2.6	70.9±1.8 (CNN-PFF, TF-C)	+1.2
RW-Waist	50	70.3±2.7 (CNN-PFF)	69.9±1.6 (CNN-PFF, LFR)	-0.4	76.0±2.0 (TS2Vec Encoder, TF-C)	+5.7
RW-Waist	100	72.3±3.9 (TS2Vec Encoder)	70.7±1.1 (CNN-PFF, LFR)	-1.6	76.8±3.0 (TS2Vec Encoder, TF-C)	+4.5
RW-Waist	200	74.5±1.4 (CNN-PFF)	72.1±1.9 (CNN-PFF, TNC)	-2.4	76.4±1.2 (CNN-PFF, TF-C)	+1.9
RW-Waist	max	74.1±5.2 (TS2Vec Encoder)	74.5±0.6 (CNN-PFF, TNC)	+0.4	**76.8±1.2 (CNN-PFF, TF-C)**	+2.8
UCI	1	57.6±9.3 (ResNet-SE-5)	53.3±8.7 (TS2Vec Encoder, TNC)	-4.3	42.8±4.3 (TS2Vec Encoder, TF-C)	-14.9
UCI	5	77.1±2.3 (ResNet-SE-5)	68.2±2.6 (TS2Vec Encoder, TNC)	-8.9	63.2±6.9 (CNN-PFF, TF-C)	-14.0
UCI	10	76.7±0.9 (ResNet-SE-5)	71.3±2.6 (TS2Vec Encoder, DIET)	-5.4	75.7±3.7 (TS2Vec Encoder, TF-C)	-0.9
UCI	25	79.9±1.6 (ResNet-SE-5)	77.3±2.1 (TS2Vec Encoder, LFR)	-2.5	84.1±1.7 (TS2Vec Encoder, TF-C)	+4.2
UCI	50	85.5±2.1 (ResNet-SE-5)	80.5±0.2 (TS2Vec Encoder, LFR)	-5.0	85.7±2.0 (CNN-PFF, TF-C)	+0.2
UCI	100	91.5±1.6 (ResNet-SE-5)	81.5±2.1 (TS2Vec Encoder, DIET)	-10.0	87.7±0.6 (CNN-PFF, TF-C)	-3.8
UCI	200	92.4±2.0 (TS2Vec Encoder)	84.3±0.2 (TS2Vec Encoder, DIET)	-8.1	88.8±1.0 (CNN-PFF, TF-C)	-3.6
UCI	max	**94.7±0.7 (ResNet-SE-5)**	85.3±1.4 (TS2Vec Encoder, DIET)	-9.4	89.7±1.7 (CNN-PFF, TF-C)	-5.1
WISDM	1	48.3±12.2 (IMU Transformer)	58.2±10.4 (ResNet-SE-5, TNC)	+9.9	46.6±6.3 (TS2Vec Encoder, TF-C)	-1.7
WISDM	5	71.6±1.6 (ResNet-SE-5)	68.7±5.6 (TS-TCC Encoder, TNC)	-2.9	63.7±8.3 (TS2Vec Encoder, TF-C)	-7.9
WISDM	10	71.7±1.5 (IMU Transformer)	68.2±4.0 (TS-TCC Encoder, TNC)	-3.5	68.4±5.6 (RNN, TF-C)	-3.3
WISDM	25	77.6±0.6 (TS-TCC Encoder)	72.8±2.2 (TS-TCC Encoder, TNC)	-4.7	77.5±0.8 (CNN-PFF, TF-C)	-0.1
WISDM	50	75.2±2.9 (TS-TCC Encoder)	75.6±0.9 (TS2Vec Encoder, LFR)	+0.5	84.0±2.3 (CNN-PFF, TF-C)	+8.9
WISDM	100	78.2±1.1 (CNN-PFF)	76.8±1.9 (TS2Vec Encoder, LFR)	-1.4	84.7±2.2 (CNN-PFF, TF-C)	+6.4
WISDM	200	81.8±1.6 (ResNet-SE-5)	79.1±1.0 (RNN, LFR)	-2.7	85.8±1.4 (CNN-PFF, TF-C)	+4.0
WISDM	max	**87.3±0.9 (CNN-PFF)**	85.2±0.3 (TS-TCC Encoder, DIET)	-2.0	87.2±1.4 (CNN-PFF, TF-C)	-0.1
Summary		72.5±3.4	68.2±3.1	**≥+1: 5** **≤-1: 39**	71.1±3.3	**≥+1: 18** **≤-1: 22**

Frozen SSL models with TF-C can surpass supervised performance in specific scenarios, particularly on RW-Waist with 100 samples per class, while achieving comparable results on KH and WISDM datasets. However, in extremely scarce data regimes (under 25 samples per class), different encoder-SSL combinations emerge as optimal: TS2Vec and TS-TCC Encoder with DIET/TNC, and ResNet-SE-5 with TNC demonstrate competitive performance when compared with TF-C, suggesting that TF-C's architectural complexity may be less beneficial when very limited labeled data is available for adaptation at freeze.

This happened more frequently than what was observed under full fine-tuning scenarios where TF-C dominates more consistently, highlighting that the optimal SSL technique depends on the refinement strategy and data availability. The freezing approach shows particular promise for scenarios where computational efficiency is prioritized, as it maintains competitive performance while avoiding expensive refinement of the entire encoder.

APPENDIX D
SUPPLEMENTARY BENCHMARK TOOL

We developed an interactive benchmark analysis tool as supplementary material to provide the research community with access to our experimental results. This web-based tool offers dynamic exploration of all performance data across the complete parameter space, including:

- **Complete Results Access**: All combinations of encoders, techniques, and datasets stratified by samples per class (1 to maximum)
- **Advanced Filtering**: Select specific techniques (Supervised, TF-C, TNC, LFR, DIET), encoders, datasets, and refinement strategies
- **Interactive Visualization**: Performance trend charts across different data regimes
- **Custom Comparisons**: Side-by-side analysis of selected combinations with highlighting of best performers
- **User Data Integration**: Capability to add new experimental results and compare against our benchmark
- **Data Export**: Download filtered results for further analysis

The tool presents performance metrics including maximum, minimum, and average values alongside complete learning curves. This enables direct benchmarking and facilitates future comparative studies in smartphone-based HAR. The tool and source code for all experiments are publicly available on GitHub[3]

APPENDIX E
COMPUTATIONAL COSTS

In order to address the computational costs of SSL techniques, we measured the inference time for each encoder trained under all strategies, including supervised and SSL techniques. From this, we report in Table 15 the average inference time per sample at the test set, the amount of model parameters, and the amount of Multiply-Accumulate operations (MACs) per inference.

Table 15 shows that the inference time for most encoders ranges from 0.62 to 1.46 milliseconds, whereas the TS2Vec Encoder requires between 49 and 99 milliseconds, with 76M MACs per inference. Despite its longer inference time, the TS2Vec Encoder does not have the largest number of

[3]https://github.com/H-IAAC/benchmarking-encoders-ssl-har

TABLE 15. Average inference time measured from 20 rounds per training strategy on an NVIDIA RTX 5000 Ada Generation 32GB GPU. We measured the inference time for the entire RealWorld-Thigh test partition and divided by the number of samples. The values for the supervised strategy and the TNC, DIET, and LFR techniques were grouped together, as they have the same number of parameters and MACs and a similar inference time, shown by the low standard deviation. Applying the TF-C technique results in roughly doubling the inference time and amount of MAC operations. The inference time of the TS2Vec encoder is up to 50 times greater than the other encoders. The rows are sorted according to inference time.

Strategy	Encoder	Inference Time (ms)	Params (M)	MACs (M)
Supervised/ TNC/DIET/LFR	RNN	0.62 ± 0.03	0.17	4.08
	CNN-PFF	0.71 ± 0.03	0.15	1.67
	ResNet-SE-5	0.81 ± 0.05	0.14	3.07
	IMU Transformer	0.89 ± 0.05	0.23	6.97
	TS-TCC Encoder	0.96 ± 0.08	3.33	5.07
	TS2Vec Encoder	48.52 ± 2.30	0.68	38.16
TFC	RNN	0.77 ± 0.04	0.52	8.34
	CNN-PFF	0.87 ± 0.05	0.59	3.65
	ResNet-SE-5	1.10 ± 0.04	0.39	6.27
	IMU Transformer	1.22 ± 0.07	0.57	14.06
	TS-TCC Encoder	1.46 ± 0.07	7.35	10.83
	TS2Vec Encoder	98.90 ± 1.98	1.54	76.50

TABLE 16. Encoder performance and detailed misclassification rates per class. Columns show training strategy, accuracy, and the percentage of misclassified samples per class. (-) indicates classes not present. The table is sorted by highest accuracy per encoder for each dataset. The most commonly confused classes are sit and stand, but there is also confusion between stair up, stair down, and walk.

				Misclassification Rate (%)					
Dataset	Encoder	Strat	Acc(%)	Sit	Stand	Walk	St.Up	St.Down	Run
KH	TS2Vec	TF-C	**90.3**	16.7	8.3	29.2	4.2	0.0	0.0
	CNN-PFF	TF-C	81.2	0.0	100.0	0.0	4.2	8.3	0.0
	ResNet-SE-5	DIET	80.6	100.0	0.0	4.2	8.3	4.2	0.0
	IMU Trans.	LFR	77.1	100.0	0.0	0.0	8.3	29.2	0.0
	RNN	TF-C	74.3	29.2	50.0	16.7	20.8	37.5	0.0
	TS-TCC	TF-C	73.6	0.0	100.0	4.2	54.2	0.0	0.0
MS	TS2Vec	LFR	**97.5**	0.6	4.0	1.7	5.1	2.8	1.1
	CNN-PFF	TF-C	95.0	0.6	11.9	2.3	6.8	4.5	4.0
	ResNet-SE-5	LFR	93.8	9.0	4.5	2.3	16.9	2.3	2.3
	TS-TCC	LFR	92.8	12.4	9.0	1.1	9.6	4.0	6.8
	RNN	TF-C	92.0	2.3	5.6	1.7	16.4	20.3	1.7
	IMU Trans.	TF-C	89.7	2.3	7.9	6.8	33.3	8.5	2.8
RW-Thigh	ResNet-SE-5	TF-C	**82.8**	28.6	7.9	9.3	21.3	15.7	20.3
	CNN-PFF	TF-C	81.5	15.1	25.3	14.5	22.8	12.2	21.1
	TS2Vec	TF-C	81.2	21.1	29.2	3.9	20.5	14.1	23.8
	RNN	TF-C	81.0	14.9	25.5	9.1	27.5	12.6	24.4
	TS-TCC	LFR	74.9	4.6	79.7	5.2	22.8	18.4	20.1
	IMU Trans.	LFR	72.2	4.3	44.7	32.1	21.5	18.0	46.4
RW-Waist	TS2Vec	TF-C	**82.5**	16.4	33.6	6.5	15.0	11.6	21.8
	RNN	TF-C	82.1	18.1	15.5	12.0	26.2	11.6	23.8
	CNN-PFF	TF-C	81.8	29.6	19.4	16.0	19.2	7.9	16.9
	IMU Trans.	LFR	80.1	32.6	5.8	18.5	23.1	15.3	24.1
	ResNet-SE-5	TF-C	79.2	43.3	20.8	11.8	15.3	10.9	22.5
	TS-TCC	TF-C	76.5	46.3	24.1	9.7	21.1	8.8	31.0
UCI	CNN-PFF	TF-C	**96.2**	10.9	5.1	0.7	2.2	0.0	-
	TS2Vec	TF-C	96.1	7.2	11.6	0.0	0.7	0.0	-
	ResNet-SE-5	DIET	95.9	9.4	10.1	0.7	0.0	0.0	-
	RNN	TF-C	94.9	2.9	15.9	2.9	3.6	0.0	-
	TS-TCC	TF-C	94.3	3.6	15.9	0.0	8.0	0.7	-
	IMU Trans.	TF-C	92.3	11.6	13.0	5.1	1.4	7.2	-
WISDM	CNN-PFF	TF-C	**91.2**	28.4	5.8	0.5	-	-	0.5
	TS2Vec	TF-C	90.7	19.7	15.7	1.2	-	-	0.6
	RNN	TF-C	89.2	32.6	6.0	3.5	-	-	3.5
	IMU Trans.	TF-C	89.2	24.6	12.6	5.2	-	-	0.6
	TS-TCC	TF-C	88.9	22.9	19.4	0.9	-	-	1.1
	ResNet-SE-5	TF-C	86.9	41.5	8.6	2.0	-	-	0.5

parameters; instead, the TS-TCC Encoder contains the most parameters among the architectures.

The results presented in Section VI demonstrate that pairing TF-C with CNN-PFF consistently yields superior accuracy. Although the TF-C strategy doubles the computational overhead, its integration with CNN-PFF remains competitive against alternative encoders, such as ResNet-SE-5, IMU Transformer, and TS-TCC, combined with other pre-training frameworks like TNC, DIET, and LFR.

APPENDIX F
CONFUSION PATTERNS

We investigate class confusion patterns by analyzing the best-performing encoders for each dataset, detailing the misclassification rate per class of each combination. For each configuration, we selected the single best model under the full fine-tuning refinement, regardless of the training strategy employed. Table 16 details the results for each class and indicates which training strategies achieved the highest accuracies for each encoder and dataset. Fig. 6 shows the t-SNE visualizations for each row in Table 16, enabling a qualitative analysis of class separation.

The most frequent confusions occur between *sit* and *stand*, as well as among *stair up*, *stair down*, and *walk*. While these confusion patterns are recurrent, their intensity varies depending on the dataset and encoder, as discussed in Section VI. For instance, on RW-Waist, TS2Vec and RNN achieve similar accuracies (82.5% and 82.1%), yet they differ in how they misclassify *stand*, *walk*, and *stair up*.

APPENDIX G
ACRONYMS

Acronym	Meaning
ADF	Augmented Dickey-Fuller.
AE	Autoencoder.
BYOL	Bootstrap Your Own Latent.
CAE	Convolutional Autoencoder.
CMC	Contrastive Multiview Coding.
CNN	Convolutional Neural Network.
CoCoA	Cross mOdality COntrastive leArning.
CPC	Contrastive Predictive Coding.
DACL	Domain-Agnostic Contrastive Learning.
DIET	Datum IndEx as Target.
ECG	Electrocardiogram.
FC	Fully Connected.
FCN	Fully Convolutional Network.
GeLU	Gaussian Error Linear Unit.
GMC	Geometric Multimodal Contrastive.
GRU	Gated Recurrent Unit.
HAR	Human Activity Recognition.
HHAR	Heterogeneity Dataset for Human Activity Recognition.
IMU	Inertial Measurement Unit.
KH	KuHar Dataset.
LFR	Learning from Randomness.
LR	Learning Rate.
LSTM	Long Short-Term Memory.
MAC	Multiply-Accumulate.
MAE	Masked Autoencoder.

MHA	Multi-Head Attention.
MLP	Multi-Layer Perceptron.
MS	MotionSense Dataset.
MTSS	Multi-Task Self-Supervised.
NNCLR	Nearest-Neighbour Contrastive Learning of visual Representations.
NT-Xent	Normalized Temperature-Scaled Cross Entropy.
PCA	Principal Component Analysis.
REBAR	Retrieval-Based Reconstruction.
ReLU	Rectified Linear Unit.
ResNet	Residual Network.
RNN	Recurrent Neural Network.
RQ	Research Question.
RW	RealWorld Dataset.
RW-Thigh	RealWorld Thigh Dataset.
RW-Waist	RealWorld Waist Dataset.
SCARF	Self-Supervised Contrastive Learning using Random Feature Corruption.
SE	Squeeze-and-Excitation.
SimSiam	Simple Siamese Network.
SimCLR	Simple framework for Contrastive Learning of visual Representations.
SOTA	State Of The Art.
SL	Supervised Learning.
SSL	Self-Supervised Learning.
STab	Self-supervised learning for Tabular Data.
TF-C	Time-Frequency Consistency.
TNC	Temporal Neighborhood Coding.
TS-TCC	Time-Series representation learning framework via Temporal and Contextual Contrasting.
t-SNE	t-distributed Stochastic Neighbor Embedding.
UCI	University of California Irvine.

REFERENCES

[1] A. Logacjov, "Self-supervised learning for accelerometer-based human activity recognition: A survey," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 8, no. 4, pp. 1–42, Nov. 2024.

[2] H. Haresamudram, I. Essa, and T. Plötz, "Assessing the state of self-supervised human activity recognition using wearables," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 6, no. 3, pp. 1–47, Sep. 2022.

[3] C. A. Ronao and S.-B. Cho, "Human activity recognition with smartphone sensors using deep learning neural networks," *Expert Syst. Appl.*, vol. 59, pp. 235–244, Oct. 2016.

[4] D. Garcia-Gonzalez, D. Rivero, E. Fernandez-Blanco, and M. R. Luaces, "Deep learning models for real-life human activity recognition from smartphone sensor data," *Internet Things*, vol. 24, Dec. 2023, Art. no. 100925.

[5] O. Napoli, D. Duarte, P. Alves, D. H. P. Soto, H. E. de Oliveira, A. Rocha, L. Boccato, and E. Borin, "A benchmark for domain adaptation and generalization in smartphone-based human activity recognition," *Sci. Data*, vol. 11, no. 1, p. 1192, Nov. 2024.

[6] G. P. C. P. da Luz, O. O. Napoli, J. V. Delgado, A. R. Rocha, L. Boccato, and E. Borin, "An evaluation of temporal neighborhood coding variants in smartphone-based human activity recognition," in *Proc. Brazilian Conf. Intell. Syst.* Cham, Switzerland: Springer, 2024, pp. 82–94.

[7] S. Liu, T. Kimura, D. Liu, R. Wang, J. Li, S. Diggavi, M. Srivastava, and T. Abdelzaher, "FOCAL: Contrastive learning for multimodal time-series sensing signals in factorized orthogonal latent space," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 47309–47338.

[8] B. E. R. da Silva, O. O. Napoli, J. V. Delgado, A. R. Rocha, L. Boccato, and E. Borin, "Impact of pre-training datasets on human activity recognition with contrastive predictive coding," in *Proc. Brazilian Conf. Intell. Syst.* Cham, Switzerland: Springer, 2024, pp. 306–320.

[9] N. Hecker, O. O. Napoli, J. Delgado, A. R. Rocha, L. Boccato, and E. Borin, "An analysis of time-frequency consistency in human activity recognition," in *Proc. Brazilian Conf. Intell. Syst.* Cham, Switzerland: Springer, 2024, pp. 66–81.

[10] M. A. Xu, A. Moreno, H. Wei, B. M. Marlin, and J. M. Rehg, "REBAR: Retrieval-based reconstruction for time-series contrastive learning," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 15440–15464.

[11] Y. Sui, T. Wu, J. C. Cresswell, G. Wu, G. Stein, X. S. Huang, X. Zhang, and M. Volkovs, "Self-supervised representation learning from random data projectors," in *Proc. Int. Conf. Learn. Represent.*, 2023, pp. 6792–6814.

[12] H. Qian, T. Tian, and C. Miao, "What makes good contrastive learning on small-scale wearable-based tasks?" in *Proc. 28th ACM SIGKDD Conf. Knowl. Discovery Data Mining*, Aug. 2022, pp. 3761–3771.

[13] S. Ek, R. Presotto, G. Civitarese, F. Portet, P. Lalanda, and C. Bettini, "Comparing self-supervised learning techniques for wearable human activity recognition," *CCF Trans. Pervasive Comput. Interact.*, vol. 7, no. 3, pp. 324–341, Sep. 2025.

[14] S. Wan, L. Qi, X. Xu, C. Tong, and Z. Gu, "Deep learning models for real-time human activity recognition with smartphones," *Mobile Netw. Appl.*, vol. 25, no. 2, pp. 743–755, Apr. 2019.

[15] M. Ronald, A. Poulose, and D. S. Han, "ISPLInception: An inception-ResNet deep learning architecture for human activity recognition," *IEEE Access*, vol. 9, pp. 68985–69001, 2021.

[16] S. Yao, S. Hu, Y. Zhao, A. Zhang, and T. Abdelzaher, "DeepSense: A unified deep learning framework for time-series mobile sensing data processing," in *Proc. 26th Int. Conf. World Wide Web*, Apr. 2017, pp. 351–360.

[17] M. Ayub and E.-S. M. El-Alfy, "G-SwinHAR: Swin transformer for smartphone-based human activity recognition using gramian angular field," in *Proc. Int. Conf. Neural Inf. Process.*, M. Mahmud, M. Doborjeh, K. Wong, A. C. S. Leung, Z. Doborjeh, and M. Tanveer, Eds., Cham, Switzerland: Springer, 2024, pp. 212–226.

[18] N. Dangol and S. Mahfuz, "Hybrid CNN-LSTM-GRU with attention for human activity recognition," *Proc. Comput. Sci.*, vol. 265, pp. 99–106, Apr. 2025.

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, Jun. 2016, pp. 770–778.

[20] S. Mekruksavanich and A. Jitpattanakul, "Deep residual network for smartwatch-based user identification through complex hand movements," *Sensors*, vol. 22, no. 8, p. 3094, Apr. 2022.

[21] S. Ha and S. Choi, "Convolutional neural networks for human activity recognition using multiple accelerometer and gyroscope sensors," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Jul. 2016, pp. 381–388.

[22] Y. Shavit and I. Klein, "Boosting inertial-based human activity recognition with transformers," *IEEE Access*, vol. 9, pp. 53540–53547, 2021.

[23] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. U. Kaiser, and I. Polosukhin, "Attention is all you need," in *Proc. Adv. Neural Inf. Process. Syst.*, vol. 30, 2017. [Online]. Available: https://papers.nips.cc/paper_files/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html

[24] Z. Yue, Y. Wang, J. Duan, T. Yang, C. Huang, Y. Tong, and B. Xu, "TS2Vec: Towards universal representation of time series," in *Proc. AAAI Conf. Artif. Intell.*, vol. 36, 2022, pp. 8980–8987.

[25] E. Eldele, M. Ragab, Z. Chen, M. Wu, C. K. Kwoh, X. Li, and C. Guan, "Time-series representation learning via temporal and contextual contrasting," in *Proc. 30th Int. Joint Conf. Artif. Intell.*, Aug. 2021, pp. 2352–2359.

[26] Z. Wang, W. Yan, and T. Oates, "Time series classification from scratch with deep neural networks: A strong baseline," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, May 2017, pp. 1578–1585.

[27] S. Tonekaboni, D. Eytan, and A. Goldenberg, "Unsupervised representation learning for time series with temporal neighborhood coding," in *Proc. Int. Conf. Learn. Represent.*, 2021. [Online]. Available: https://openreview.net/forum?id=8qDwejCuCN

[28] X. Zhang, Z. Zhao, T. Tsiligkaridis, and M. Žitnik, "Self-supervised contrastive pre-training for time series via time-frequency consistency," in *Proc. Adv. Neural Inf. Process. Syst.*, 2022, pp. 3988–4003.
[29] R. Balestriero, "Unsupervised learning on a DIET: Datum IndEx as target free of self-supervision, reconstruction, projector head," 2023, *arXiv:2302.10260*.
[30] Y. Wang, H. Liang, and B. Zhai, "Temporal neighborhood based self-supervised pre-training model for sleep stages classification," in *Proc. 15th Int. Conf. Bioinf. Biomed. Technol.*, May 2023, pp. 149–155.
[31] C. Guo, Y. Zhang, Y. Chen, C. Xu, and Z. Wang, "Modality consistency-guided contrastive learning for wearable-based human activity recognition," *IEEE Internet Things J.*, vol. 11, no. 12, pp. 21750–21762, Jun. 2024.
[32] E. Weisstein. (2026). *Bonferroni Correction*. Accessed: Fed. 24, 2026. [Online]. Available: https://mathworld.wolfram.com/BonferroniCorrection.html
[33] O. Napoli and E. Borin, "On domain generalization for human activity recognition with mix-based methods," in *Proc. Eur. Symp. Artif. Neural Netw., Comput. Intell. Mach. Learn.*, 2025, pp. 491–496.
[34] Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo, "Swin transformer: Hierarchical vision transformer using shifted windows," in *Proc. IEEE/CVF Int. Conf. Comput. Vis. (ICCV)*, Oct. 2021, pp. 9992–10002.
[35] J. E. Hu, Y. Shen, P. Wallis, Z. Allen-Zhu, Y. Li, S. Wang, W. Chen, and C. Weizhu, "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Represent.*, 2022. [Online]. Available: https://openreview.net/forum?id=nZeVKeeFYf9
[36] E. Casilari, J. A. Santoyo-Ramón, and J. M. Cano-García, "UMAFall: A multisensor dataset for the research on automatic fall detection," *Proc. Comput. Sci.*, vol. 110, pp. 32–39, Jul. 2017.
[37] H. Haresamudram, I. Essa, and T. Plötz, "Contrastive predictive coding for human activity recognition," *Proc. ACM Interact., Mobile, Wearable Ubiquitous Technol.*, vol. 5, no. 2, pp. 1–26, Jun. 2021.
[38] T. Chen, "A simple framework for contrastive learning of visual representations," in *Proc. 37th Int. Conf. Mach. Learn.*, vol. 1, Jul. 2024, pp. 1597–1607.
[39] Y. Bian, X. Ju, J. Li, Z. Xu, D. Cheng, and Q. Xu, "Multi-patch prediction: Adapting LLMs for time series representation learning," 2024, *arXiv:2402.04852*.

GUSTAVO P. C. P. DA LUZ received the B.S. degree in production engineering from the University of Brasília (UnB), Brasília, Brazil, in 2022. He is currently pursuing the M.Sc. degree in computer science with the Institute of Computing, University of Campinas (UNICAMP), Campinas, Brazil. He has been a Researcher with the Discovery Laboratory (UNICAMP), since 2023, working on deep learning architectures for smart city applications. Since 2024, he has also been a Researcher with the Hub for Artificial Intelligence and Cognitive Architectures (H.IAAC), where he works on self-supervised learning and representation learning for human activity recognition. His previous experience includes research positions with Samsung (2021–2023), and the UIoT Laboratory (UnB, 2020–2022). His research interests include deep learning, computer vision, self-supervised learning, the Internet of Things, edge AI, and smart city systems.

DARLINNE H. P. SOTO received the master's degree in computer science from the Institute of Computing, University of Campinas (UNICAMP), Brazil, in 2024, where he is currently pursuing the Ph.D. degree with the Institute of Computing. He is a Researcher with the Hub for Artificial Intelligence and Cognitive Architecture (H.IAAC), exploring the application of topological autoencoders to human activity recognition. His current research is focused on self-supervised learning, human activity recognition, foundational models, and its application on resource-constrained devices.

OTÁVIO O. NAPOLI received the master's degree in computer science from the Institute of Computing, UNICAMP, Brazil, in 2019, under the guidance of Prof. Edson Borin, where he is currently pursuing the Ph.D. degree with the Institute of Computing. He has researched side channels in the process of dynamic binary translation. He has participated in various projects, including dynamic binary translation, high-performance computing, cloud computing, compiler optimizations, and artificial intelligence. He is also with the Hub of Artificial Intelligence and Cognitive Architectures (H.IAAC), UNICAMP, focusing on knowledge representation, aiming to create compact representations for activities derived from inertial sensors of mobile devices using self-supervised learning. He is also involved in projects with large Brazilian companies, such as Petrobras, using machine learning to efficiently compute seismic attributes.

ANDERSON ROCHA (Fellow, IEEE) is a Full Professor of artificial intelligence and digital forensics with the Institute of Computing, University of Campinas (UNICAMP), Brazil. He is the Head of the Artificial Intelligence Laboratory (Recod.ai), UNICAMP. He was the former Director of the Institute, from 2019 to 2023. He is an elected affiliate of Brazilian Academy of Sciences (ABC) and Brazilian Academy of Forensic Sciences (ABC). He is an Elected Member (three-term) and the former Chair (two-term) of the IEEE Information Forensics and Security Technical Committee (IFS-TC). He is a Microsoft Research, a Google Research, a Tan Chin Tuan (TCT), and an Asia Pacific Artificial Intelligence Association Faculty Fellow. He is ranked among the Top 2% of research scientists worldwide, according to PlosOne/Stanford and Research.com studies. He is a LinkedIn Top Voice in artificial intelligence. He is an IEEE Distinguished Lecturer and an IEEE Biometrics Distinguished Lecturer.

LEVY BOCCATO received the B.S. degree in computer engineering and the M.Sc. and Ph.D. degrees in electrical engineering from the University of Campinas (UNICAMP), São Paulo, Brazil, in 2008, 2010, and 2013, respectively. He is currently an Associate Professor with UNICAMP, where he conducts research with the Laboratory of Signal Processing for Communications (DSPCom). He is also a member of Brazilian Institute of Data Science (BI0S) and the Hub of Artificial Intelligence and Cognitive Architectures (H.IAAC). His main research interests include signal processing, adaptive filtering, machine learning, and brain–computer interfaces.

EDSON BORIN (Member, IEEE) is an Associate Professor and the Head of the Ph.D. and M.Sc. programs with the Institute of Computing, University of Campinas (UNICAMP). He leads the Knowledge Representation Research Group, H.IAAC (https://hiaac.unicamp.br/), where he investigates machine learning techniques for developing efficient representations for cognitive architectures on mobile devices. He also leads the Seismic Processing Foundational Models Team, focusing on representation learning methods for the development of foundational models for seismic processing tasks. He coordinates the Discovery Laboratory (https://discovery.ic.unicamp.br/); and is the author of nine patents, a technical book on assembly programming, and over 100 publications in international conferences and journals. He has supervised more than 30 postdoctoral researchers and graduate students, many of whom have received awards for their theses, dissertations, and publications.

Coordenação de Aperfeiçoamento de Pessoal de Nível Superior (CAPES) - ROR identifier: 00x0ma614